

INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS
CAMPUS MONTES CLAROS
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

CONFIGURAÇÃO E PROJETO AUTOMÁTICOS
DE ALGORITMOS APLICADOS AO PROBLEMA
JUST-IN-TIME JOB SHOP SCHEDULING

MAICON PEDRO MACEDO LELES
ORIENTADOR: PROF. DR. TADEU KNEWITZ ZUBARAN
COORIENTADOR: DR. ARTUR FERREIRA BRUM

Montes Claros

Junho de 2026

MAICON PEDRO MACEDO LELES

CONFIGURAÇÃO E PROJETO AUTOMÁTICOS
DE ALGORITMOS APLICADOS AO PROBLEMA
JUST-IN-TIME JOB SHOP SCHEDULING

Monografia apresentada ao Curso de Graduação em Ciência da Computação do Instituto Federal do Norte de Minas Gerais – Campus Montes Claros, como requisito parcial para a obtenção do grau de Bacharel em Ciência da Computação.

ORIENTADOR: PROF. DR. TADEU KNEWITZ ZUBARAN
COORIENTADOR: DR. ARTUR FERREIRA BRUM

Montes Claros

Junho de 2026

© 2026, Maicon Pedro Macedo Leles.
Todos os direitos reservados.

Leles, Maicon Pedro Macedo
xxxx Configuração e projeto automáticos de algoritmos
aplicados ao problema *Just-In-Time Job Shop Scheduling*
/ Maicon Pedro Macedo Leles. — Montes Claros, 2026
xxiii, 70 f. : il. ; 29cm

Monografia (graduação) — Instituto Federal do
Norte de Minas Gerais

Orientador: Prof. Dr. Tadeu Knewitz Zubaran
Coorientador: Dr. Artur Ferreira Brum

1. Projeto Automático de Algoritmos.
2. Configuração Automática de Algoritmos.
3. Problemas de Agendamento. 4. Heurísticas. I. Título.

CDU xxxxxxxx

[Folha de Aprovação]

Quando a secretaria do Curso fornecer esta folha,
ela deve ser digitalizada e armazenada no disco em formato gráfico.

Se você estiver usando o `pdflatex`,
armazene o arquivo preferencialmente em formato PNG
(o formato JPEG é pior neste caso).

Se você estiver usando o `latex` (não o `pdflatex`),
terá que converter o arquivo gráfico para o formato EPS.

Em seguida, acrescente a opção `approval={nome do arquivo}`
ao comando `\ppgccufmg`.

Aos meus pais, Eulidiane e Pedro, com amor, por fazerem dos meus sonhos os seus.

Agradecimentos

Aos meus pais, Eulidiane Guedes Macedo Leles e Pedro Leles Junior, pelo amor, carinho, apoio incondicional, confiança, sacrifícios e por serem exemplos de pessoas em quem me inspiro todos os dias da minha vida. Vocês compõem a base de tudo que sou e são minha maior motivação para continuar conquistando meus objetivos.

À minha irmã, Sthefane Macedo Leles, pelo amor, amizade e companheirismo que proporcionou à minha vida. Sua doçura e alegria me inspiram.

Ao meu orientador, Prof. Dr. Tadeu Knewitz Zubaran, pela excelente orientação, ideias, paciência, confiança no meu trabalho, conselhos e por compartilhar um pouco de sua experiência comigo. Sem sua orientação, este trabalho não seria possível.

Ao meu coorientador, Dr. Artur Ferreira Brum, pelas opiniões, ideias e pela sua disponibilidade, mesmo à distância.

Aos queridos amigos e colegas de curso, pelas conversas, momentos de descontração, apoio, e por tornarem toda essa experiência mais leve. Foi um prazer estar rodeado de pessoas extremamente competentes no que fazem.

Ao meu querido amigo Ivanderlei Mendes Silva Filho, por ter me acompanhado desde o primeiro dia de aula até o encerramento desta graduação. Agradeço pelo companheirismo, competência, confiança e por todo o trabalho que realizamos juntos. Conviver com sua criatividade tornou este projeto possível, e sua alegria constante trouxe leveza a todo esse processo.

A todos os funcionários do IFNMG - Campus Montes Claros, em especial aos professores do curso de Ciência da Computação.

*“If more of us valued food and cheer and song above hoarded gold, it would be a
merrier world.”*

(J. R. R. Tolkien)

Resumo

A alocação eficiente de recursos para a execução de processos é fundamental em diferentes indústrias de transporte, distribuição e produção. Podemos modelar esses processos de alocação como problemas de otimização. Um caso específico desses problemas é o *Just-In-Time Job Shop Scheduling* (JITJSS), no qual as tarefas a serem executadas possuem prazos de entrega, e finalizar essas tarefas fora do seu prazo gera penalidades de adiantamento e atraso. Com isso, nosso objetivo é minimizar a soma dessas penalidades. Sabendo que o JITJSS é um problema fortemente NP-difícil, uma solução exata é computacionalmente inviável na prática. Por isso, uma alternativa para resolver problemas desse tipo é utilizar heurísticas, ou seja, estratégias que obtêm soluções em um tempo viável, mas sem garantia de qualidade. Porém, a construção de heurísticas gera, para o projetista, a dificuldade de testar manualmente diferentes combinações de estratégias para encontrar uma combinação que componha uma heurística eficiente. Por isso, o processo se torna tedioso, custoso, dificilmente reproduzível e inconclusivo. Para mitigar esses problemas, a comunidade científica desenvolveu diversas ferramentas e técnicas para a Configuração Automática de Algoritmos (do inglês, *Automatic Algorithm Configuration* – AAC) e Projeto Automático de Algoritmos (do inglês, *Automatic Algorithm Design* – AAD). Essas ferramentas e técnicas sistematizam e automatizam a parte mecânica do desenvolvimento de heurísticas, permitindo ao projetista criar algoritmos mais eficientes. De fato, a eficácia das técnicas de AAC e AAD na geração de algoritmos de nível estado da arte já está comprovada pelos resultados de diversos trabalhos na literatura. Porém, a aplicação dessas técnicas ao JITJSS ainda não foi plenamente explorada. A partir disso, o objetivo deste trabalho é aplicar a AAC e o AAD ao JITJSS, propondo um novo espaço de projeto algorítmico e gerando, de forma automática, novos algoritmos para o problema. Para tanto, estruturou-se um espaço de busca parametrizado via gramática livre de contexto. O processo de configuração convergiu para uma arquitetura baseada em Busca Tabu Iterada, validando a relevância da vizinhança original proposta, `troca_crit`, que esteve presente em 80% das soluções elites. Nos testes experimentais realizados sobre 72 instâncias de teste da literatura, o algoritmo gerado igualou ou superou o melhor resultado

conhecido em aproximadamente 72,2% dos cenários. Ademais, o método demonstrou alta eficiência temporal, localizando as soluções finais em uma média de 21,1 segundos. Os resultados obtidos validam a efetividade dos componentes construídos e cancelam o projeto automático como uma alternativa robusta à calibração manual de heurísticas para problemas complexos de agendamento.

Palavras-chave: Projeto Automático de Algoritmos, Configuração Automática de Algoritmos, Problemas de Agendamento, Heurísticas.

Abstract

Efficient resource allocation for process execution is fundamental in various industries, such as transport, distribution, and manufacturing. We can model these allocation processes as optimization problems. A specific case is the *Just-In-Time Job Shop Scheduling* (JITJSS) problem, where tasks have specific due dates, and completing them outside these deadlines generates earliness or tardiness penalties. The objective is to minimize the sum of these penalties. The JITJSS is known to be a strongly NP-Hard problem, meaning that finding an exact solution is computationally infeasible in practice. Therefore, an alternative is to use heuristics, strategies that obtain solutions in a reasonable time, although without optimality guarantees. The challenge in heuristic construction is that the designer must manually test different combinations of algorithmic components to find the most efficient one. This process is tedious, costly, difficult to reproduce, and often inconclusive. To mitigate these issues, the scientific community has developed tools and techniques for Automatic Algorithm Configuration (AAC) and Automatic Algorithm Design (AAD). These techniques systematize and automate the mechanical aspects of heuristic development, enabling designers to create more efficient algorithms. The effectiveness of AAD in generating state-of-the-art algorithms is well-documented across various domains. However, these techniques have not yet been applied to the JITJSS problem. Therefore, the objective of this work is to apply AAC and AAD techniques to JITJSS, proposing a new algorithmic design space and automatically generating new algorithms for the problem. To this end, a search space parameterized via a context-free grammar was structured. The configuration process converged to an architecture based on Iterated Tabu Search, validating the relevance of the proposed original neighborhood, `troca_crit`, which was present in 80% of the elite solutions. In experimental tests conducted on 72 literature test instances, the generated algorithm matched or surpassed the best known value in approximately 72.2% of the scenarios. Furthermore, the method demonstrated high temporal efficiency, locating the final solutions in an average of 21.1 seconds. The obtained results validate the effectiveness of the constructed components and endorse automatic design as a robust alternative to manual heuristic tuning for complex scheduling problems.

Keywords: Automatic Algorithm Design, Automatic Algorithm Configuration, Scheduling Problems, Heuristics.

Sumário

Agradecimentos	ix
Resumo	xiii
Abstract	xv
Lista de Figuras	xix
Lista de Tabelas	xxi
Lista de Siglas	xxiii
1 Introdução	1
1.1 Objetivos	4
1.1.1 Objetivo Geral	4
1.1.2 Objetivos Específicos	4
1.2 Organização do Trabalho	4
2 Referencial Teórico	5
2.1 Complexidade de Problemas	5
2.2 Problemas de Agendamento	6
2.2.1 <i>Job Shops</i> e <i>Flow Shops</i>	7
2.2.2 <i>Just-In-Time Job Shop Scheduling</i>	8
2.2.3 Grafo disjuntivo	10
2.2.4 Algoritmos para geração de agendamentos	13
2.3 Configuração Automática de Algoritmos	14
2.3.1 Vizinhanças	15
2.3.2 Meta-heurísticas	17
2.3.3 Problema de Configuração <i>offline</i> de algoritmos	20
2.3.4 Processo de configuração automática e configuradores	22
2.4 Projeto automático de algoritmos	24

2.4.1	Abordagem <i>Top-Down</i>	26
2.4.2	Abordagem <i>Bottom-Up</i>	28
3	Trabalhos Relacionados	31
3.1	<i>Just-In-Time Job Shop</i>	31
3.2	Projeto Automático de Algoritmos	33
4	Metodologia	37
4.1	Materiais	38
4.2	Atividades	39
4.3	Cronograma	39
4.3.1	Proposta de cronograma inicial	39
4.3.2	Cronograma executado	40
5	Resultados	41
5.1	Espaço de projeto construído	41
5.1.1	Soluções iniciais	42
5.1.2	Vizinhanças	42
5.1.3	Métodos de busca	47
5.1.4	Agendadores	50
5.2	Configuração automática	50
5.2.1	Gramática e parâmetros	50
5.2.2	Instâncias de treino	54
5.2.3	Execução do irace	54
5.3	Análise	55
5.3.1	Validade do processo de configuração	55
5.3.2	Teste das configurações	58
6	Conclusão	63
	Referências Bibliográficas	65

Lista de Figuras

2.1	Agendamento de uma instância do JITJSS	10
2.2	Grafo disjuntivo gerado a partir da instância de exemplo	11
2.3	Grafo direcionado acíclico que representa uma sequência viável	12
2.4	Agendamento ativo que minimiza o <i>makespan</i>	13
2.5	Mínimos locais em relação à função objetivo no espaço de soluções	16
2.6	Transformações de mínimos locais por funções de perturbação.	18
2.7	Fluxo de execução de um algoritmo configurador	22
2.8	Fluxo de um configurador no processo de AAD	26
2.9	Gramática livre de contexto na forma Backus-Naur para geração de um algoritmo de busca	28
5.1	Exemplo de modificação realizada pela vizinhança relax.	42
5.2	Exemplos dos dois tipos principais de vizinhanças: troca e remoção-inserção.	43
5.3	Exemplo de modificação de soluções pela vizinhança troca_penal.	44
5.4	Exemplos das modificações da vizinhança insere_penal para um operação adiantada.	46
5.5	Exemplo de modificações vizinhas geradas pela vizinhança troca_crit.	47
5.6	Gramática livre de contexto para geração dos algoritmos	51
5.7	Gráfico da variância da distância de Gower entre as configurações elite pelas iterações do irace.	57

Lista de Tabelas

2.1	Instância de um problema de agendamento com 3 tarefas com 3 operações cada	9
2.2	Estratégias disponíveis para cada componente da meta-heurística ILS . . .	27
4.1	Proposta de cronograma inicial	39
4.2	Cronograma de atividades realizado	40
5.1	Componentes estruturais do espaço de projeto de algoritmos.	41
5.2	Espaço de busca de parâmetros do irace.	53
5.3	Configurações Elite Finais	55
5.4	Resumo dos resultados das configurações elite.	59
5.5	Resultados da configuração C1 para as instâncias de teste de 10 tarefas. . .	59
5.6	Resultados da configuração C1 para as instâncias de teste de 15 tarefas. . .	60
5.7	Resultados da configuração C1 para as instâncias de teste de 20 tarefas. . .	61
5.8	Comparação individual entre este trabalho e algoritmos da literatura. . . .	62

Lista de Siglas

AAC	Configuração Automática de Algoritmos (do inglês, <i>Automatic Algorithm Configuration</i>)	3
AAD	Projeto Automático de Algoritmos (do inglês, <i>Automatic Algorithm Design</i>)	3
ACO	Otimização por Colônia de Formigas (do inglês, <i>Ant Colony Optimazation</i>)	33
EDD	Data de Entrega mais Próxima (do inglês, <i>Earliest Due Date</i>)	50
FSP	<i>Flow Shop Problem</i>	3
GT	Giffler-Thompson	14
HFS	<i>Hybrid Flow Shop</i>	3
ILS	Busca Local Iterada (do inglês, <i>Iterated Local Search</i>)	18
JITJSS	<i>Just-In-Time Job Shop Scheduling</i>	1
JSS	<i>Job Shop Scheduling</i>	7
LS	Busca Local (do inglês, <i>Local Search</i>)	51
NPFSP	<i>Non Permutation Flow Shop Problem</i>	3
PFSP	<i>Permutation Flow Shop Problem</i>	3
PFSP-WT	<i>Permutation Flow Shop Problem</i> com atraso ponderado	34
SA	Recozimento Simulado (do inglês, <i>Simulated Annealing</i>)	25
SAT	Problema da Satisfabilidade Proposicional (do inglês, <i>Boolean Satisfiability Problem</i>)	33
TFT	<i>Total Flow Time</i>	34
TS	Busca Tabu (do inglês, <i>Tabu Search</i>)	51
UBQP	<i>Unconstrained Binary Quadratic Programming</i>	3
VNS	Busca em Vizinhança Variável (do inglês, <i>Variable Neighborhood Search</i>)	18

Capítulo 1

Introdução

O agendamento eficiente de recursos para a execução de tarefas cumpre um papel fundamental em ambientes de produção e manufatura, bem como em indústrias de transporte e distribuição. Nesse sentido, diferentes cenários podem exigir a otimização de diferentes objetivos. A partir disso, podemos modelar tais cenários de agendamento de tarefas como problemas de otimização combinatória. A flexibilidade desses problemas nos permite acoplar diversas características à modelagem, de forma que ela represente fielmente um cenário real [Pinedo, 2008]. A partir dessa abordagem, podemos representar uma infinidade de processos de gerenciamento de recursos.

Um caso específico de problema de agendamento é o problema *Just-In-Time Job Shop Scheduling* ([JITJSS](#)). Além da alocação de máquinas para executar tarefas, esse problema define prazos para cada etapa de uma tarefa e caso alguma dessas etapas seja finalizada fora do seu prazo de entrega, ela gera uma penalidade de adiantamento ou de atraso. O objetivo ao resolver esse problema é encontrar uma alocação que minimize essas penalidades. Esse problema foi proposto pela primeira vez por Baptiste et al. [2008], que também propuseram as instâncias do problema, juntamente com os primeiros limites superiores e inferiores para essas instâncias.

A relevância do [JITJSS](#) reside na sua capacidade de modelar cenários nos quais a finalização de processos dentro de um prazo é essencial. Por exemplo, em uma malha ferroviária, alocar os tempos em que cada locomotiva ocupa cada estação é essencial para o fluxo eficiente dos trens. Podemos modelar as paradas como tarefas que precisam de recursos (estações) para serem executadas e que duram um determinado tempo. Além disso, cada parada de trem possui um prazo desejável para encerrar e liberar a estação. Dessa maneira, queremos alocar as estações minimizando ao máximo os desvios desses prazos de liberação. Para mais detalhes sobre o problema de alocação de trens, veja Liu & Kozan [2011] e Flamini & Pacciarelli [2008].

Considere, como outro exemplo de aplicação, os procedimentos cirúrgicos de um

hospital. Para a execução desses procedimentos, há recursos limitados disponíveis que serão alocados por um determinado tempo para suas diferentes fases. Os processos de cada paciente possuem tempos de duração diferentes e uma ordem de execução predefinida. Também existem prazos desejáveis para que cada processo seja finalizado. Dessa forma, uma maneira de alocar tais processos hospitalares aos recursos disponíveis, minimizando as execuções fora do prazo, otimiza os processos hospitalares descritos. Veja Pham & Klinkert [2008] para mais detalhes sobre esse problema. Embora essas aplicações do mundo real possuam suas próprias complexidades, elas ilustram a estrutura e a importância dos problemas de agendamento com restrições de tempo, como o [JITJSS](#).

Além de representar características de cenários do mundo real, o problema [JITJSS](#) é fortemente NP-Difícil [Du & Leung, 1990]. Para problemas dessa classe, não existem soluções exatas eficientes, a menos que $P=NP$ [Cormen et al., 2022]. Em vista disso, uma opção é tentar encontrar soluções exatas, embora ineficientes. O problema nessa abordagem é que encontrar soluções exatas se torna computacionalmente inviável para instâncias grandes. Uma outra opção é trocar a otimalidade das soluções por um algoritmo mais eficiente. Nesse sentido, existem algoritmos aproximados que encontram soluções garantidamente próximas do ótimo. Por exemplo, um algoritmo 1,2-aproximado para um problema de minimização sempre encontra soluções até 20% piores do que a solução ótima. O problema nessa abordagem é que desenvolver tais estratégias exige um entendimento pleno das características do problema e demonstrações robustas da aproximação construída. Por esses motivos, uma opção recorrente para resolver problemas difíceis é utilizar heurísticas [Kleinberg & Tardos, 2005].

Heurísticas são soluções que não garantem a otimalidade da solução em troca de obter soluções em tempo viável. Por não terem o compromisso com a otimalidade, desenvolver estratégias heurísticas livra o projetista dos problemas no desenvolvimento de métodos exatos e aproximados [Kleinberg & Tardos, 2005].

Podemos encontrar na literatura diversas aplicações de heurísticas ao problema [JITJSS](#). A título de exemplo, temos algoritmos evolucionários [Araujo et al., 2009; Dos Santos et al., 2010; Yang et al., 2012], a busca em grandes vizinhanças [Laborie & Godard, 2007; Abderrazzak et al., 2022] e a busca em vizinhança variável [Wang & Li, 2014; Ahmadian et al., 2021].

A eficácia de uma heurística, no entanto, depende de uma série de decisões de projeto que os projetistas precisam realizar. Essas decisões são baseadas em ciclos de desenvolvimento e validação das estratégias desenvolvidas. Porém, mesmo com o processo de desenvolvimento finalizado, algumas escolhas de implementação permanecem em aberto, e o projetista pode expor essas decisões aos usuários através de parâmetros do algoritmo. Dessa forma, o usuário pode definir a combinação de parâmetros que

obtem os melhores resultados para o seu objetivo. O problema é que encontrar tal combinação de parâmetros pode se tornar uma tarefa extremamente custosa à medida que o número de parâmetros cresce [Birattari & Kacprzyk, 2009].

Nesse sentido, definimos uma configuração de um algoritmo como uma atribuição de valores a cada parâmetro. A partir disso, temos que o processo de realizar a busca da configuração ideal manualmente possui alguns problemas. Além de ser um processo custoso e tedioso para o indivíduo responsável, é um processo muito propenso a erros, dificilmente reproduzível e, mesmo com muito esforço do configurador, ainda é um processo inconclusivo pois não nos dá argumentos suficientes de que a configuração encontrada é realmente boa. Para resolver esse problema, a comunidade científica formalizou a configuração de algoritmos e desenvolveu diversos métodos de Configuração Automática de Algoritmos (do inglês, *Automatic Algorithm Configuration* – AAC) [Birattari & Kacprzyk, 2009]. Esses métodos nos permitem explorar o espaço de configurações sistematicamente e, com isso, encontrar configurações comprovadamente boas.

Utilizando as técnicas de Configuração Automática de Algoritmos (do inglês, *Automatic Algorithm Configuration*) (AAC), fica mais fácil encontrar boas configurações para um algoritmo. Ainda assim, primeiro precisamos construir um algoritmo para conseguirmos configurá-lo. Para isso, precisamos testar diversas combinações de componentes algorítmicos para encontrarmos bons algoritmos que resolvam um certo problema. Esse processo, se realizado de forma manual, sofre dos mesmos problemas da configuração manual de algoritmos. Levando a ideia do AAC um passo adiante e com o objetivo de otimizar esse processo, a comunidade científica propôs expor totalmente as decisões de projeto de algoritmos às técnicas de AAC. Dessa maneira, seria possível explorar sistematicamente o espaço de algoritmos para um problema. A partir dessa ideia, a comunidade científica desenvolveu diversos métodos e conceitos de Projeto Automático de Algoritmos (do inglês, *Automatic Algorithm Design* – AAD) [Stützle & López-Ibáñez, 2018].

Com o avanço das técnicas de Projeto Automático de Algoritmos (do inglês, *Automatic Algorithm Design*) (AAD), esses métodos vêm sendo utilizados para gerar, automaticamente, novos algoritmos estado da arte para diferentes problemas de otimização combinatória, como o Problema da Satisfabilidade Proposicional (do inglês, *Boolean Satisfiability Problem* – SAT) [KhudaBukhsh et al., 2016], *Unconstrained Binary Quadratic Programming* (UBQP) [De Souza & Ritt, 2018a,b], *Flow Shop Problem* (FSP) [Brum & Ritt, 2018], *Permutation Flow Shop Problem* (PFSP) [Marmion et al., 2013; Pagnozzi & Stützle, 2019], *Hybrid Flow Shop* (HFS) [Alfaro-Fernández et al., 2020] e *Non Permutation Flow Shop Problem* (NPFSP) [Brum et al., 2022].

A partir de todas essas ideias, percebemos o sucesso na aplicação do AAD em diver-

sos problemas de otimização, incluindo vários problemas de agendamento. Contudo, ainda não há na literatura um trabalho que aplique as técnicas de [AAD](#) para resolver o problema [JITJSS](#). Por isso, considerando a relevância do [JITJSS](#) e a lacuna na literatura, o presente trabalho parte da hipótese de que a aplicação das técnicas de projeto automático tenha a capacidade de gerar novos algoritmos que superem o estado da arte atual.

1.1 Objetivos

1.1.1 Objetivo Geral

O presente trabalho tem como objetivo aplicar as técnicas de [AAC](#) e [AAD](#) na resolução do problema [JITJSS](#) e, com isso, contribuir para um melhor entendimento sobre o problema e sobre a aplicação das técnicas de [AAC](#) e [AAD](#) em problemas de agendamento.

1.1.2 Objetivos Específicos

Para atingir o nosso objetivo geral, estabelecemos os seguintes objetivos específicos:

1. Definir o espaço de projeto de algoritmos.
2. Propor pelo menos um componente heurístico específico para o [JITJSS](#), a fim de incrementar o espaço de projeto de algoritmos.
3. Aplicar as técnicas de [AAC](#) e [AAD](#) sobre o espaço de projeto definido anteriormente, adotando as boas práticas metodológicas da literatura para garantir a robustez e a validade do processo de configuração.
4. Avaliar a qualidade das estratégias obtidas pela aplicação das técnicas de [AAC](#) e [AAD](#).

1.2 Organização do Trabalho

Além deste capítulo introdutório, o presente trabalho está organizado da seguinte forma: O Capítulo [2](#) detalha os conceitos que embasam o presente trabalho; O Capítulo [3](#) realiza uma análise da literatura relacionada ao presente trabalho; O Capítulo [4](#) explica os métodos e materiais que serão utilizados na realização deste trabalho; O Capítulo [5](#) expõe o trabalho realizado e os resultados obtidos; E, por fim, o Capítulo [6](#) consolida os resultados obtidos e apresenta os possíveis trabalhos futuros.

Capítulo 2

Referencial Teórico

2.1 Complexidade de Problemas

Uma das motivações do presente trabalho é a existência de problemas muito difíceis de resolver em tempo viável. Nesse sentido, precisamos desenvolver e propor abordagens capazes de encontrar soluções para tais problemas. Mas, primeiramente, precisamos entender como classificar problemas em relação a sua dificuldade.

Kleinberg & Tardos [2005] definem como eficientes algoritmos que possuem tempo de execução polinomial. A partir dessa ideia, podemos definir a classe de problemas P . Essa classe contém todos os problemas para os quais existe algum algoritmo polinomial, isto é, um algoritmo com tempo de execução $O(p(n))$, sendo p uma função polinomial, e n o tamanho da instância dada ao algoritmo [Sipser, 2013].

Pela definição de P , os problemas fora dessa classe são aqueles para os quais não conhecemos um algoritmo polinomial. Uma parte desses problemas pertence à classe NP. Essa classe contém todos os problemas verificáveis em tempo polinomial. Um problema é verificável em tempo polinomial se, ao recebermos uma solução para o problema, conseguimos verificar sua validade em tempo polinomial. Para a definição completa da classe NP, veja Kleinberg & Tardos [2005].

Além da classe NP, a caracterização da classe de problemas NP-difíceis representa um dos avanços mais importantes do estudo da complexidade. Um problema X é NP-difícil se existir uma redução polinomial de qualquer problema Y da classe NP para X . Essa definição implica que, se encontrarmos um algoritmo polinomial para qualquer problema NP-difícil, seremos capazes de resolver todos os problemas em NP em tempo polinomial. Por fim, se um problema pertence à classe NP e também é NP-difícil, ele pertence à classe de problemas NP-completos. Para mais detalhes sobre as definições de redução polinomial, NP-difícil e NP-completo, veja Kleinberg & Tardos [2005] e Garey & Johnson [1990].

2.2 Problemas de Agendamento

Problemas de agendamento envolvem a alocação de recursos para a execução de tarefas, buscando otimizar uma ou mais variáveis. Em problemas de *Shop Scheduling*, os recursos são máquinas destinadas à execução de tarefas (do inglês, *Jobs*) [Pinedo, 2008]. Diferentes problemas de *Shop* possuem características diversas que servem à modelagem de vários problemas do mundo real. Algumas dessas características são comuns à maioria dos problemas de *Shop*.

Uma dessas características comuns é que cada tarefa consiste em um conjunto de operações, cada uma relacionada à máquina que deve processá-la. A ordem em que as operações devem passar pelas máquinas pode ser mais ou menos flexível de acordo com o problema analisado. Além disso, cada tarefa possui tempos de processamento associados a cada máquina que faz parte de sua sequência de processamento [Pinedo, 2008].

Outra característica mais específica é a existência de pesos associados a cada tarefa que determinam a importância do processamento de uma tarefa em relação às demais. Alguns problemas também incorporam prazos de entrega para cada tarefa, gerando penalidades caso uma tarefa finalize seu processamento fora desse prazo. Outros exemplos de características incluem a recirculação que, em um problema de agendamento, indica que as tarefas podem precisar passar pela mesma máquina mais de uma vez durante seu processamento. Além disso, temos também as restrições de precedência que agem sobre a ordem e determinam se alguma fase de processamento precisa esperar outra fase terminar para iniciar [Pinedo, 2008].

Para formalizar essas características e considerando que os problemas de *shop* trabalham com um número finito de tarefas e máquinas, definimos $J = \{J_1, J_2, \dots, J_n\}$ e $M = \{1, 2, \dots, m\}$ como sendo, respectivamente, os conjuntos de tarefas e máquinas. Cada tarefa J_i possui um conjunto com n_i operações $O_i = \{o_{i1}, o_{i2}, \dots, o_{in_i}\}$. Cada uma dessas operações o_{ik} possui um tempo de processamento associado p_{ik} e uma máquina associada $M(o_{ik})$.

Precisamos, neste ponto, introduzir uma maneira de quantificar a qualidade das soluções de um problema chamada de função objetivo. Essa função mapeia todo o conjunto de soluções S de um problema para valores reais, $f : S \rightarrow \mathbb{R}$. Essa ferramenta nos permite comparar soluções pela sua qualidade [Talbi, 2009]. Além disso, o presente trabalho foca apenas em problemas de minimização, isto é, quanto menor o custo de uma solução, melhor é a solução. Por isso, estabelecemos que, por conveniência, sendo s_1 e s_2 soluções para um dado problema, $f(s_1) < f(s_2)$ significará que a solução s_1 é melhor do que a solução s_2 .

Em um problema de otimização, dada uma instância, temos como objetivo encon-

trar a solução que possua o melhor valor de f para essa instância. Em um mesmo problema de agendamento, essa função objetivo pode quantificar diferentes características das soluções. Uma das funções objetivo mais comuns é o *makespan* (C_{max}). Definimos o *makespan* como o tempo de término da última tarefa a finalizar seu processamento [Pinedo, 2008].

Outros exemplos de função objetivo em problemas de agendamento com prazos de entrega incluem o atraso máximo (L_{max}), que consiste no maior tempo de atraso entre as operações. Ademais, temos também o atraso total ponderado ($\sum w_i T_i$) que consiste no somatório de todos os tempos de atraso de forma que cada atraso possui um peso associado.

A seguir, definiremos brevemente *Job Shops* e *Flow Shops*. Em seguida, definiremos em mais detalhes o problema [JITJSS](#) focando em suas particularidades como um problema de agendamento e, por fim, explicaremos uma forma de organizar a resolução de problemas de *Job Shop* em duas etapas.

2.2.1 *Job Shops* e *Flow Shops*

Em diversas linhas de produção, as tarefas passam pelas mesmas máquinas na mesma ordem. Essa característica define o problema de agendamento [FSP](#). Nesse problema, dado um conjunto de n tarefas e m máquinas, temos que cada tarefa possui m operações. Dessa forma, a sequência de operações de cada tarefa segue a ordem das máquinas, ou seja, a k -ésima operação da tarefa i , o_{ik} , é processada na máquina k para todo $1 \leq i \leq n$ e $1 \leq k \leq m$.

Uma variação comum do [FSP](#) é o [PFSP](#), que surge ao adicionarmos a restrição de que a sequência de processamento das tarefas deve ser a mesma em todas as máquinas. Nesse caso, uma solução para o problema é representada por uma única permutação dos índices das tarefas $\pi = \{\pi(1), \pi(2), \dots, \pi(n)\}$.

Quando cada tarefa segue uma ordem definida de processamento nas máquinas, mas não necessariamente a mesma, temos um problema de *Job Shop Scheduling* ([JSS](#)). Nesse tipo de problema, um dos objetivos mais comuns é otimizar o *makespan*. Definimos o problema de forma que cada tarefa J_i possui um número n_i de operações. Cada operação o_{ik} de uma tarefa requer uma máquina $M(o_{ik}) \in M$ para seu processamento. Diante disso, para cada $M_u \in M$, denotamos por $\mathcal{O}(M_u)$ o conjunto de operações que requerem a máquina M_u . Além disso, cada tarefa possui uma ordem fixa de passagem pelas máquinas e uma máquina não pode executar mais de uma operação simultaneamente.

Em relação à complexidade desses problemas, temos que eles são naturalmente difíceis. Por exemplo, Garey et al. [1976] provaram que o [FSP](#), mesmo reduzido a três

máquinas, é NP-difícil. Ademais, minimizar o *makespan* no **JSS** também é NP-difícil, mesmo restrito a 3 máquinas com até 2 operações por tarefa [Lawler et al., 1993].

2.2.2 *Just-In-Time Job Shop Scheduling*

Baptiste et al. [2008] propuseram o problema **JITJSS** com o objetivo de analisar um modelo mais complexo de *Job Shop* com penalidades de adiantamento e atraso com múltiplas máquinas. O problema surge como uma opção para modelar um ambiente em que finalizar processos precocemente gera, por exemplo, um custo de armazenamento do resultado, enquanto finalizar processos tardiamente gera penalidades de atraso.

Assim, podemos definir o problema de forma que, dado um conjunto de tarefas $J = \{J_1, \dots, J_n\}$ e um conjunto de máquinas $M = \{M_1, \dots, M_m\}$. Cada tarefa J_i possui um conjunto de m operações $O_i = \{o_{i1}, \dots, o_{im}\}$, sendo o_{ik} a k -ésima operação da tarefa i . Cada operação o_{ik} de uma tarefa requer uma máquina $M(o_{ik}) \in M$ onde será executada. Desta forma, para cada $M_u \in M$, denotamos por $\mathcal{O}(M_u)$ o conjunto de operações que requerem a máquina M_u [Baptiste et al., 2008].

Além do mais, cada operação o_{ik} possui uma data de entrega d_{ik} e um tempo de processamento p_{ik} . Seja C_{ik} o tempo de término da operação o_{ik} . Caso $C_{ik} < d_{ik}$, a operação o_{ik} incorre em uma penalidade na forma $\alpha_{ik}E_{ik}$, onde α_{ik} é o coeficiente de adiantamento da operação e E_{ik} é o tempo de adiantamento dado por $\max(d_{ik} - C_{ik}, 0)$. Por outro lado, caso $C_{ik} > d_{ik}$, a operação o_{ik} terá uma penalidade $\beta_{ik}T_{ik}$, onde β_{ik} é o coeficiente de atraso e T_{ik} é o tempo de atraso definido como $\max(C_{ik} - d_{ik}, 0)$. [Baptiste et al., 2008].

Podemos modelar o problema matematicamente da seguinte forma [Baptiste et al., 2008]:

$$z = \min \sum_{i=1}^n \sum_{k=1}^m \alpha_{ik}E_{ik} + \beta_{ik}T_{ik} \quad (2.1)$$

sujeito a

$$C_{i1} - p_{i1} \geq 0, \quad J_i \in J \quad (2.2)$$

$$C_{ik} \leq C_{i,k+1} - p_{i,k+1}, \quad J_i \in J, o_{ik} \in O_i - \{o_{im}\} \quad (2.3)$$

$$C_{ik} \geq C_{jh} + p_{ik} \vee C_{jh} \geq C_{ik} + p_{jh}, \quad M_u \in M, o_{ik}, o_{jh} \in \mathcal{O}(M_u) \quad (2.4)$$

$$E_{ik} \geq d_{ik} - C_{ik}, \quad J_i \in J, o_{ik} \in O_i \quad (2.5)$$

$$T_{ik} \geq C_{ik} - d_{ik}, \quad J_i \in J, o_{ik} \in O_i \quad (2.6)$$

$$E_{ik} \geq 0, \quad J_i \in J, o_{ik} \in O_i \quad (2.7)$$

$$T_{ik} \geq 0, \quad J_i \in J, o_{ik} \in O_i \quad (2.8)$$

Primeiramente, a Equação 2.1 define como função objetivo minimizar o somatório das penalidades de adiantamento e de atraso. A partir disso, a Equação 2.2 define a restrição que impede o tempo de início de processamento de ser negativo. Além disso, a Equação 2.3 refere a restrição que define uma ordem de precedência fixa para a execução das operações de cada tarefa, de forma que o início do processamento de uma operação depende do término de sua predecessora. Do mesmo modo, a Equação 2.4 restringe que uma máquina não pode executar duas operações ao mesmo tempo. Ademais, as equações 2.5 e 2.6 modelam os tempos de adiantamento e atraso, respectivamente. Por fim, as equações 2.7 e 2.8 asseguram que os tempos de adiantamento e de atraso não sejam negativos.

Para exemplificar, considere a instância do JITJSS detalhada na Tabela 2.1. Cada linha da tabela representa uma tarefa e cada coluna k representa a k -ésima operação de uma tarefa. Cada célula contém uma tupla $(m; p; d; \alpha; \beta)$, sendo m o índice da máquina requerida pela operação, p o tempo de processamento da operação, d a data de entrega da operação e α e β , respectivamente, os coeficientes de adiantamento e atraso de cada operação. A partir desta instância, podemos gerar o agendamento da Figura 2.1. Nessa figura, as barras da mesma cor representam operações da mesma tarefa.

Tabela 2.1. Instância de um problema de agendamento com 3 tarefas com 3 operações cada

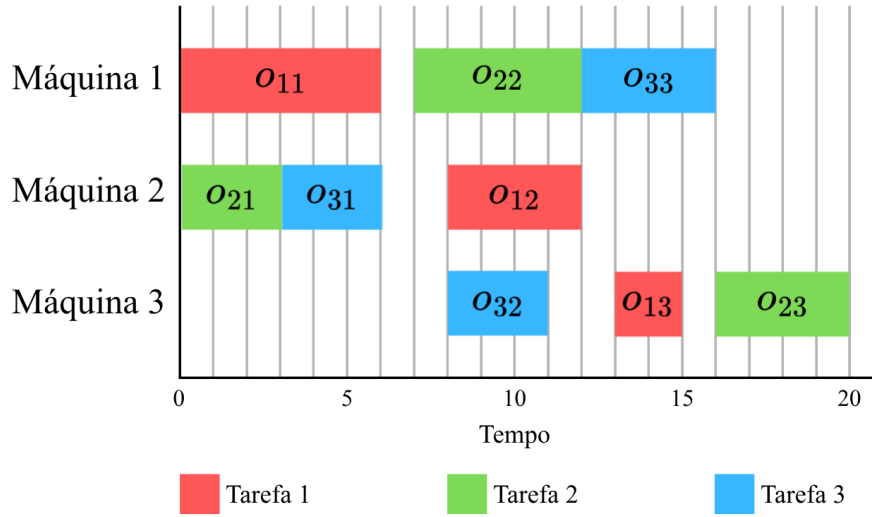
Tarefa	Operação 1	Operação 2	Operação 3
J_1	(1; 6; 6; 0, 2; 0, 15)	(2; 4; 12; 0, 4; 0, 7)	(3; 2; 15; 0, 3; 0, 4)
J_2	(2; 3; 7; 0, 1; 0, 15)	(1; 5; 16; 0, 3; 0, 2)	(3; 4; 20; 0, 5; 0, 1)
J_3	(2; 3; 4; 0, 7; 0, 35)	(3; 3; 11; 0, 9; 0, 1)	(1; 4; 16; 0, 3; 0, 6)

Fonte: Os Autores.

Ao observarmos os prazos de entrega de cada operação, podemos calcular as penalidades resultantes desse agendamento. Dessa forma, temos as operações o_{21} e o_{22}

adiantadas, a operação o_{31} atrasada e o restante das operações entregues no prazo. Com isso, calculamos a função objetivo com os coeficientes e os tempos de atraso e adiantamento, obtendo $z = 4 \times 0,1 + 4 \times 0,3 + 2 \times 0,35 = 2,3$.

Figura 2.1. Agendamento de uma instância do JITJSS



Fonte: Os Autores.

Dada a descrição e a modelagem do problema, podemos observar com mais detalhes o processo de encontrar uma solução para ele. Primeiramente, precisamos gerar uma sequência válida a partir da instância. Perceba que a instância do problema define uma ordem fixa para as operações de uma mesma tarefa, mas a ordem em que as operações utilizarão uma mesma máquina fica em aberto. Dessa forma, gerar uma sequência válida significa determinar a ordem de execução das operações em cada máquina. Para entender o processo de forma detalhada, precisamos introduzir o conceito de grafo disjuntivo na modelagem de problemas de agendamento.

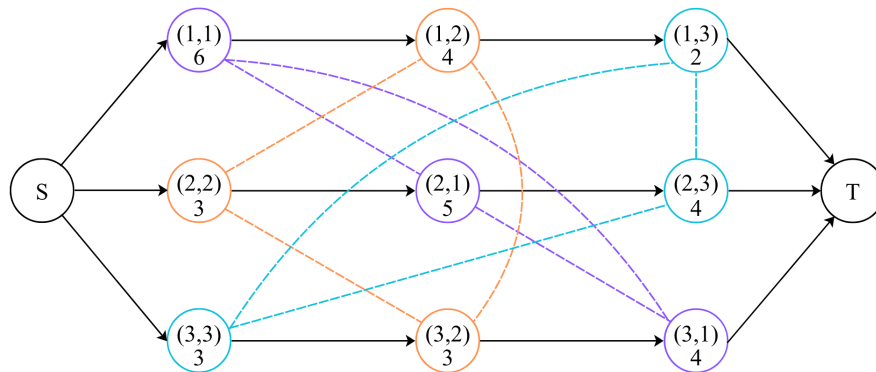
2.2.3 Grafo disjuntivo

Considere o grafo $G = (V, X, Y)$. Nesse grafo, para cada operação o_{ik} de uma tarefa J_i , existe um vértice $u \in V$ representado por um par ordenado (i, j) , sendo i o índice da tarefa e $j = M(o_{ik})$ o índice da máquina que processa essa operação. Além disso, o grafo possui dois conjuntos de arestas. As arestas conjuntivas X e as arestas disjuntivas Y . As arestas conjuntivas são direcionadas e representam a ordem de processamento das operações de uma tarefa. Elas são dadas pela instância e existem apenas entre operações da mesma tarefa e, se a aresta (u, v) , $u, v \in V$, existe, a operação u deve preceder a operação v . Enquanto isso, as arestas disjuntivas não são direcionadas e ligam operações que competem pela mesma máquina, isto é, se a aresta $\{u, v\}$,

$u, v \in V$, existe, as operações u e v utilizam a mesma máquina. Adicionamos também dois vértices s e t . O vértice s possui n arestas de saída na forma (s, u) para cada vértice u que representa a primeira operação de uma tarefa. Por outro lado, o vértice t possui n arestas de entrada na forma (u, t) para cada vértice u que representa a última operação de alguma tarefa. Definimos também uma função de pesos $w : V \rightarrow \mathbb{N}$ que mapeia cada vértice para um valor inteiro não negativo. Nesse âmbito, $w(s) = w(t) = 0$ [Pinedo, 2008].

Considere novamente a instância do JITJSS detalhada na Tabela 2.1. A partir dessa instância, podemos gerar o grafo disjuntivo da Figura 2.2. Nessa figura, as arestas sólidas representam as arestas conjuntivas, enquanto as arestas tracejadas representam as arestas disjuntivas. De acordo com a definição do grafo disjuntivo para um problema de agendamento, podemos identificar as operações que utilizam uma mesma máquina. Na Figura em questão, os vértices em roxo representam as operações da máquina 1, os vértices em laranja representam a máquina 2 e os vértices em azul representam a máquina 3.

Figura 2.2. Grafo disjuntivo gerado a partir da instância de exemplo

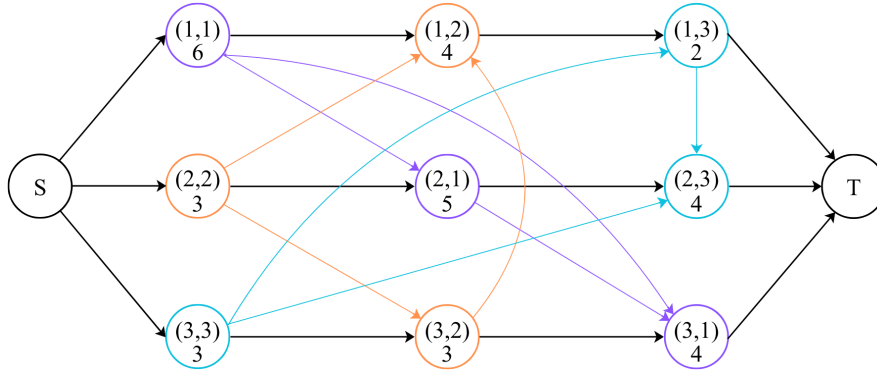


Fonte: Os Autores.

Dessa forma, temos que, dada uma instância do problema, podemos gerar o grafo disjuntivo correspondente. Como mencionamos anteriormente, as arestas direcionadas do grafo disjuntivo implicam uma ordem de processamento das operações conectadas. Além disso, gerar uma sequência significa determinar a ordem de execução das operações em cada máquina. Por conseguinte, podemos definir que gerar uma sequência a partir de um grafo disjuntivo é o processo de selecionar direções para todas as arestas disjuntivas. Ao definir direções para todas as arestas disjuntivas, fazemos uma seleção completa e obtemos um grafo direcionado. Entretanto, para que a sequência seja viável, além da seleção completa, o grafo resultante precisa ser acíclico, pois um ciclo no grafo implicaria uma ordem de processamento impossível, na qual uma operação precederia a si mesma. Para a demonstração formal, veja Pinedo [2008].

Podemos ver na Figura 2.3 o mesmo grafo do exemplo anterior, mas com a seleção completa já aplicada. Dessa forma, observando o grafo, sabemos que a máquina 3 executará as operações (3, 3), (1, 3) e (2, 3) nessa ordem.

Figura 2.3. Grafo direcionado acíclico que representa uma sequência viável



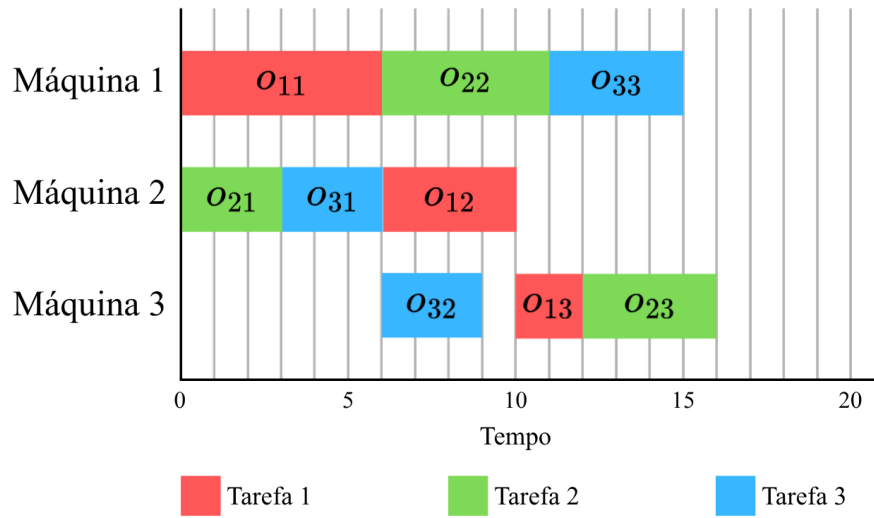
Fonte: Os Autores.

Após encontrar uma sequência viável, precisamos avaliar a qualidade dessa sequência. Nesse sentido, uma das propriedades do grafo disjuntivo é que, após realizar a seleção das arestas disjuntivas, o caminho máximo do vértice S ao T é igual ao *makespan* mínimo para essa seleção [Pinedo, 2008]. Por esse motivo, se nosso objetivo for minimizar o *makespan*, dada uma sequência, podemos avaliar sua qualidade de forma ótima em tempo linear.

Por exemplo, o caminho máximo do grafo da Figura 2.3 é composto pelos vértices (1, 1), (1, 2), (1, 3) e (2, 3), com peso total de 16. Podemos observar o agendamento que gera esse *makespan* na Figura 2.4. O termo caminho crítico define esse percurso máximo, visto que as operações que o compõem impedem qualquer movimentação sem impacto no *makespan*. Nesse sentido, nenhuma operação pertencente ao caminho crítico pode iniciar mais cedo sem que a sequência seja alterada.

Outra vantagem que temos ao minimizar o *makespan*, sabendo o caminho crítico, é saber exatamente quais operações geram o *makespan* de uma solução e, consequentemente, onde o agendamento exige modificações para alterar o *makespan*. Essa informação auxilia o processo de minimização do *makespan*, reduzindo o espaço de busca por melhorias.

No entanto, se o nosso objetivo for minimizar as penalidades de adiantamento e atraso, não temos uma forma tão simples e rápida de gerar um agendamento ótimo a partir de uma sequência. O agendamento representado na Figura 2.4 demonstra que agendar as operações o mais cedo possível não significa minimizar as penalidades, pois esse processo tende a gerar penalidades de adiantamento. Esse agendamento, por exemplo, gera uma penalidade total de 8,4, das quais 7,7 correspondem a penalidades

Figura 2.4. Agendamento ativo que minimiza o *makespan*

por adiantamento. Ademais, poderíamos atrasar o início de algumas operações para que as penalidades fossem menores, o que nos levaria ao agendamento da Figura 2.1, que é ótimo para essa sequência específica. Veja a seção 3.1 para uma revisão bibliográfica que inclui as formas de gerar agendamentos utilizadas na literatura do JITJSS.

2.2.4 Algoritmos para geração de agendamentos

Nesta seção, apresentamos formas de gerar uma sequência viável por meio de exemplos de algoritmos. Nesse sentido, a literatura classifica procedimentos que constroem um agendamento do zero como construtivos. Estes algoritmos, de forma geral, precisam selecionar operação por operação a ser agendada até que todas sejam agendadas. Chamamos de regras de despacho as diferentes formas de selecionar qual operação agendar entre um conjunto de operações [Pinedo, 2008]. Tais algoritmos construtivos são capazes de utilizar qualquer regra de despacho internamente. Dessa forma, podemos definir um algoritmo que gera um agendamento baseado apenas em uma regra de despacho arbitrária.

Sejam, respectivamente, J , M e O os conjuntos de tarefas, máquinas e operações de um problema de agendamento. Dados esses conjuntos, definimos o_{ik} como sendo a k -ésima operação da tarefa i . O algoritmo mantém uma lista L de todas as operações para as quais todas as suas antecessoras já foram agendadas, isto é, se $o_{ik} \in L$, então as operações o_{ix} , nas quais $x < k$, já foram agendadas. O procedimento inicializa a lista L com a primeira operação de cada tarefa e, a cada iteração, o algoritmo seleciona uma operação $o_{ik} \in L$ para ser agendada de acordo com uma regra de despacho e adiciona

em L a operação o_{ik+1} . Ao final de $|O|$ iterações, o algoritmo conclui o agendamento de todas as operações em O [Pinedo, 2008].

Outro algoritmo um pouco mais sofisticado é o Giffler-Thompson (GT) [Giffler & Thompson, 1960] para geração de agendamentos ativos. Um agendamento é ativo quando não é possível alterar a posição de uma operação no agendamento para que ela inicie mais cedo sem que um atraso em outra operação seja causado, em outras palavras, o agendamento não possui tempo ocioso entre duas operações no qual fosse possível agendar outra operação. Por exemplo, o agendamento da Figura 2.4 é um agendamento ativo. O algoritmo GT utiliza as mesmas estruturas que o algoritmo descrito anteriormente, mas, antes de utilizar uma regra de despacho para selecionar uma operação, ele realiza filtros na lista L para que apenas operações que mantêm o agendamento ativo sejam escolhidas.

Seja $P(o_{ik}) \in \mathbb{N}$ o tempo de processamento da operação o_{ik} e $A(o_{ik}) \in O$ a operação que antecede o_{ik} na mesma máquina. Definimos o tempo de término de uma operação $C(o_{ik}) = \max(C(o_{ik-1}), C(A(o_{ik})) + P(o_{ik}))$. O algoritmo GT seleciona em L a operação o' com menor tempo de término C' . A partir dessa operação, o algoritmo cria a lista $L_1 \subseteq L$ que contém as operações o'' as quais $M(o'') = M(o')$ e $C(o'') \leq C'$. O algoritmo, então, aplica uma regra de despacho na lista L_1 para selecionar a operação o_{ik} que será agendada, remove o_{ik} de L e adiciona o_{ik+1} em L se $o_{ik+1} \in O$. Após $|O|$ iterações, o algoritmo produz um agendamento viável e ativo.

2.3 Configuração Automática de Algoritmos

No processo de desenvolver estratégias de resolução para problemas inerentemente difíceis como o JITJSS, uma das alternativas que possuímos é utilizar heurísticas. Uma heurística resolve o problema em tempo viável, mas não garante a qualidade das soluções que encontra, no sentido de que não é possível demonstrarmos que as soluções são ótimas ou que possuem algum limite de aproximação da solução ótima [Talbi, 2009].

Durante o desenvolvimento dessas estratégias, os pesquisadores perceberam que grupos de diferentes heurísticas para problemas diferentes possuem diversas características em comum. Por exemplo, toda heurística de busca possui algum tipo de construção, recombinação ou modificação de soluções dentro do seu processo de exploração do espaço de busca. Esses conceitos são independentes de características específicas de cada problema. Por outro lado, ao observarmos a aplicação desses conceitos na literatura, percebemos estratégias específicas que os autores implementam para servir às peculiaridades do problema-alvo [Zäpfel et al., 2014].

A partir dessas ideias, abstraímos os conceitos comuns entre diferentes estratégias e

definimos métodos que podemos aplicar em qualquer problema, mas que possuem lacunas que devemos preencher com os componentes específicos ao problema que estamos resolvendo. A comunidade científica chama esses métodos genéricos de meta-heurísticas [Zäpfel et al., 2014].

O desenvolvimento de meta-heurísticas eficientes em resolver um problema leva tempo e exige do projetista um conhecimento profundo das características do problema. As meta-heurísticas tendem a facilitar esse processo por meio da generalização de métodos em comum que funcionaram para diferentes problemas. Ainda assim, desenvolver uma meta-heurística eficiente é um processo custoso. Nessa perspectiva, os projetistas de meta-heurísticas precisam tomar diversas decisões de projeto, tanto de alto quanto de baixo nível. Esses projetistas podem basear essas decisões em testes exaustivos, buscando encontrar uma configuração que retorne os melhores resultados. Porém, essa abordagem é tediosa, propensa a erros e dificilmente reproduzível [Birattari & Kacprzyk, 2009].

Com o avanço e maior disponibilidade do poder computacional e visando mitigar o problema de configurar heurísticas, a comunidade científica propôs diversos métodos de AAC. Dessa forma, os pesquisadores podem executar o trabalho repetitivo e tedioso automaticamente, ganhando tempo para, por exemplo, desenvolver novas estratégias para a resolução de problemas [Birattari & Kacprzyk, 2009].

Esta seção aborda a AAC como um problema de otimização e as principais ferramentas para resolvê-lo, detalhando inicialmente os conceitos fundamentais de vizinhanças e meta-heurísticas. No entanto, considerando o objetivo do presente trabalho, não realizaremos uma revisão abrangente de todos os tipos de meta-heurísticas, mas detalharemos apenas aquelas relacionadas ao domínio do projeto. Dessa forma, para detalhes e definições sobre meta-heurísticas, nos referimos aos trabalhos de Talbi [2009] e Zäpfel et al. [2014].

2.3.1 Vizinhanças

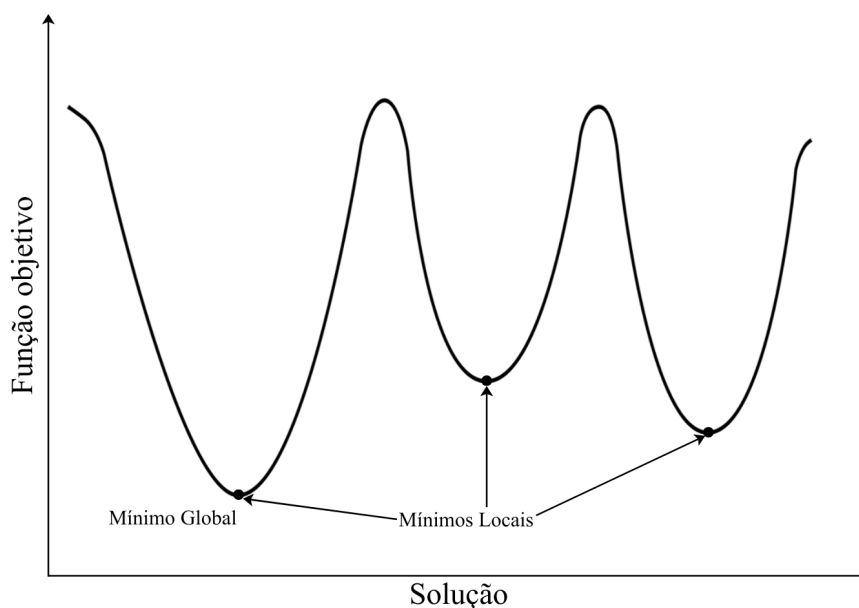
Dado o conjunto de todas as soluções S de um problema, Talbi [2009] define uma vizinhança como um mapeamento de cada solução $s \in S$ para um subconjunto de soluções $N(s) \subseteq S$. Chamamos de vizinha de s uma solução $s' \in N(s)$. O mapeamento da vizinhança modifica soluções sistematicamente para gerar novas soluções. Por isso, a vizinhança é uma componente específica do problema-alvo, isto é, sua forma e eficiência dependem exclusivamente das características do problema. Isso é verdade, pois modificar uma solução inclui manipular a representação da solução, que varia entre problemas e até mesmo entre diferentes abordagens sobre o mesmo problema [Talbi, 2009].

O projetista de uma vizinhança deve considerar algumas características durante o desenvolvimento. Uma delas é a localidade da vizinhança; essa propriedade consiste no impacto que alterações na representação de uma solução têm sobre a qualidade da solução. Se pequenas alterações na representação causam pequenas mudanças na solução, a vizinhança possui alta localidade. Por outro lado, se pequenas alterações na representação causam grandes mudanças na solução, a vizinhança possui baixa localidade. Diante disso, uma exploração do espaço de busca por meio de uma vizinhança com alta localidade é mais refinada e sistemática [Talbi, 2009].

Outro ponto importante é o tamanho da vizinhança. Gerar e explorar uma vizinhança muito grande pode ter um custo alto e, conseqüentemente, atrapalhar o processo de busca. Por outro lado, vizinhanças muito restritas e, conseqüentemente, pequenas, podem ignorar soluções promissoras no espaço de soluções. [Talbi, 2009].

Com o conceito de vizinhança estabelecido, podemos introduzir o conceito de ótimo local. Esse conceito é importante pois diversas meta-heurísticas mais sofisticadas implementam múltiplas técnicas para evitar ótimos locais. Talbi [2009] define um ótimo local como uma solução s' tal que, para toda solução vizinha $s'' \in N(s')$, temos que $f(s') \leq f(s'')$. Um ótimo global, por sua vez, é uma solução s' que, para qualquer solução s'' do espaço de busca, $f(s') \leq f(s'')$ [Talbi, 2009]. Note que, por definição, todo ótimo global é também um ótimo local, mas o inverso não é verdadeiro. A Figura 2.5 apresenta um gráfico que representa os valores de f pelo espaço de busca e indica os ótimos locais e um ótimo global.

Figura 2.5. Mínimos locais em relação à função objetivo no espaço de soluções



Fonte: Adaptado de Talbi [2009]

2.3.2 Meta-heurísticas

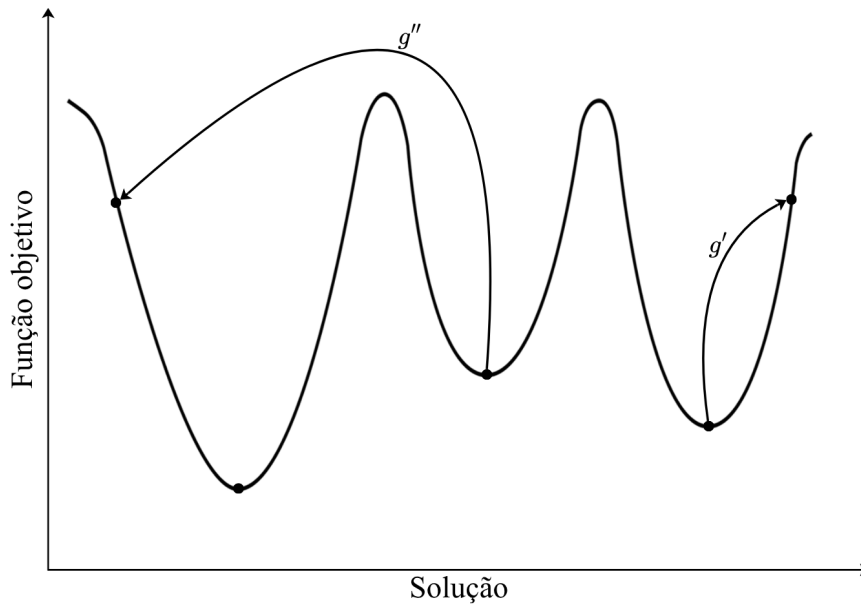
Como mencionamos anteriormente, as meta-heurísticas são estratégias de alto nível que podemos aplicar a qualquer problema. Em vista disso, a Busca Local (do inglês, Local Search – [LS](#)) é uma das meta-heurísticas mais básicas e, por isso, as meta-heurísticas mais sofisticadas utilizam seus conceitos internamente. Dada uma solução inicial s_0 , a cada iteração i , a busca local gera a vizinhança $N(s_i)$ da solução s_i . Assim, a busca local seleciona uma solução $s_{i+1} \in N(s_i)$ tal que $f(s_{i+1}) < f(s_i)$, em outras palavras, seleciona uma solução que melhora a função objetivo. Por fim, o algoritmo finaliza o processo de busca quando não existe nenhuma solução $s' \in N(s_i)$ de forma que $f(s') < f(s_i)$. Por esse motivo, a busca local não consegue escapar de ótimos locais no espaço de busca [Talbi, 2009].

Podemos realizar a exploração do espaço de busca e a seleção de vizinhos na busca local de diferentes formas. Uma delas, a seleção por melhor melhora, consiste em selecionar o melhor vizinho de uma solução a cada iteração, isto é, tendo uma solução s_i , o algoritmo seleciona a vizinha de s_i que oferece o máximo de melhoria na função objetivo em relação a $f(s_i)$. Utilizar esse tipo de seleção significa gerar e avaliar todos os vizinhos da solução a cada iteração. Esse processo pode ter um custo elevado em vizinhanças grandes [Talbi, 2009].

Outra forma de selecionar vizinhos consiste em selecionar o primeiro vizinho de s_i que melhora a função objetivo. Por isso, ao contrário da primeira opção, o algoritmo não precisa gerar nem avaliar toda a vizinhança. Isso gera uma exploração do espaço de busca mais rápida; porém, corremos o risco de que o processo ignore boas soluções simplesmente pela ordem de geração e avaliação dos vizinhos. Chamamos essa segunda estratégia de seleção por primeira melhora [Talbi, 2009].

O maior problema de uma busca local simples é a incapacidade de escapar de ótimos locais. Por esse motivo, buscas locais tendem a finalizar o processo de busca precocemente e sem realizar uma exploração satisfatória no espaço de busca. Com isso em mente, apresentamos o conceito de perturbação de uma solução. Meta-heurísticas mais complexas utilizam esse processo com o objetivo de escapar de ótimos locais. Ele geralmente se aplica aos ótimos locais e consiste em realizar alguma modificação da solução que seja significativa o suficiente para que a solução modificada se afaste do ótimo local [Talbi, 2009]. Por exemplo, no gráfico da Figura [2.6](#), temos duas funções de perturbação, g' e g'' . Nesse caso da imagem, a função g' não fez uma perturbação efetiva, pois a solução gerada não é diferente o suficiente para gerar um mínimo local diferente. Por outro lado, a função g'' gera uma solução que levará o processo de busca a um mínimo local diferente e melhor.

A Busca Local Iterada (do inglês, Iterated Local Search – [ILS](#)) é um exemplo de

Figura 2.6. Transformações de mínimos locais por funções de perturbação.**Fonte:** Os Autores.

meta-heurística que utiliza um processo de perturbação em seu processo de busca. Ela realiza, iterativamente, buscas locais com diferentes soluções iniciais para encontrar diferentes ótimos locais. Em cada iteração, ela utiliza uma perturbação do ótimo local encontrado na iteração anterior [Talbi, 2009]. Algumas características da Busca Local Iterada (do inglês, *Iterated Local Search*) ([ILS](#)) incluem a possibilidade de utilizar qualquer heurística de solução única como método de busca, isto é, o processo de encontrar um ótimo local a cada iteração é uma caixa-preta. Nesse sentido, essa caixa-preta deve receber uma solução e encontrar um ótimo local [Talbi, 2009].

Algumas meta-heurísticas renomeiam o processo de perturbação. Por exemplo, a Busca em Vizinhança Variável (do inglês, *Variable Neighborhood Search* – [VNS](#)) chama esse processo de *shake*. Essa meta-heurística se apoia na ideia de que diferentes vizinhanças de um mesmo problema exploram subespaços de busca diferentes. Mais especificamente, podemos dividir o processo de busca na Busca em Vizinhança Variável (do inglês, *Variable Neighborhood Search*) ([VNS](#)) em três fases: *Shake*, busca local e *move*. Dada uma lista de estruturas de vizinhança $N = \{N_1, N_2, \dots, N_{n-1}, N_n\}$, a cada iteração, a [VNS](#) realiza o *shake* de uma solução inicial s pela vizinhança atual $N_k \in N$, isto é, o algoritmo seleciona uma solução s' aleatoriamente da vizinhança $N_k(s)$. Após o *shake*, a [VNS](#) aplica um processo de busca local sobre a solução s' , encontrando um ótimo local s'' . Por fim, caso $f(s'') < f(s)$, a [VNS](#) realiza o *move*, ou seja, seleciona s'' como a nova solução base e redefine $k = 1$; caso contrário, mantém a solução e incrementa k , $k = k + 1$ [Talbi, 2009].

Dando continuidade às estratégias de evasão de mínimos locais, também podemos utilizar a Busca Tabu (do inglês, *Tabu Search* – TS). Essa meta-heurística é semelhante à busca local no sentido de que inicia a busca a partir de uma solução inicial s_0 e explora o espaço de busca iterativamente, selecionando novas soluções que melhoram a função objetivo. Porém, a busca tabu não encerra a busca ao chegar a um ótimo local no espaço de busca. Ao invés disso, a busca seleciona uma solução que piora a função objetivo para escapar do ótimo local. Nesse sentido, ela seleciona o vizinho com o menor valor da função objetivo, mesmo que ele não seja melhor que a solução que o gerou [Talbi, 2009].

Ao selecionar soluções que pioram a função objetivo, a busca tabu permite que soluções selecionadas em iterações anteriores sejam melhores que a solução da iteração atual. Por isso, é intuitivo pensar que a busca pode ficar presa em um ciclo infinito de pioras e melhoras da função objetivo. Porém, a busca tabu utiliza uma memória para evitar selecionar soluções repetidas. Ela implementa essa memória em uma lista tabu de movimentos realizados, e não seleciona novas soluções que desfazem esses movimentos tabu. É importante mencionar que a lista tabu mantém os estados da busca por um período denominado *tenure* [Talbi, 2009].

A implementação ideal de uma meta-heurística sobre um problema deveria obter bons resultados em todos os tipos diferentes de instâncias do problema. Porém, construir soluções que atinjam essa generalização de forma completa nem sempre é possível [Birattari & Kacprzyk, 2009]. Por exemplo, suponha um algoritmo de busca local para um certo problema X , talvez a seleção por primeira melhora seja a melhor opção para um determinado grupo de instâncias, mas, para um grupo diferente de instâncias, a seleção por melhor melhora seja ideal. O projetista precisa escolher qual tipo de seleção implementar, ou por primeira melhora, ou por melhor melhora, ou até mesmo por uma terceira forma híbrida. Esse tipo de decisão se repete constantemente no desenvolvimento de soluções. Nesses casos, o projetista pode definir e implementar uma das opções ou implementar todas e permitir ao usuário do algoritmo escolher a abordagem por meio de um parâmetro. Dessa forma, o usuário pode decidir as estratégias mais adequadas aos tipos de instâncias em questão [Birattari & Kacprzyk, 2009].

Definir parâmetros customizáveis parece resolver o problema, pois gera algoritmos mais flexíveis. Porém, mesmo assim, o projetista ainda precisou realizar uma série de decisões que compõem a base desse algoritmo flexível. Além disso, o usuário também precisará definir uma série de parâmetros de acordo com suas necessidades. Diante disso, uma das alternativas para tomar tais decisões é realizar múltiplos testes até encontrar uma configuração razoável. Porém, essa abordagem tem alguns problemas [Birattari & Kacprzyk, 2009].

Primeiramente, para que esse processo seja efetivo, o indivíduo que configurar o algoritmo precisa ter um conhecimento profundo do problema em questão, além de conhecer detalhadamente o algoritmo em si. Além disso, esse tipo de configuração pode invalidar completamente os resultados da comparação entre algoritmos. Por exemplo, considere que um projetista comparou dois algoritmos A e B . Nesse processo, ele passou 75% do seu tempo configurando o algoritmo A e os outros 25% configurando o B . Independentemente de qual algoritmo obteve os melhores resultados, o viés da configuração invalida a comparação. Ou seja, os resultados dependem da imparcialidade do projetista que configurou os algoritmos [Birattari & Kacprzyk, 2009].

É nesse contexto que surge a necessidade de métodos científicos e formais para configurar algoritmos.

2.3.3 Problema de Configuração *offline* de algoritmos

Dado o contexto de utilização da AAC, podemos definir a configuração de algoritmos como um problema de otimização. Perceba que o objetivo da configuração de algoritmos é, dado um algoritmo e instâncias de um problema, determinar os parâmetros do algoritmo que maximizam sua performance nessas instâncias [Birattari & Kacprzyk, 2009]. Nesta subseção, formalizaremos esse problema.

Chamamos o problema de *offline* pela forma como ele realiza a otimização da configuração. Em métodos *offline*, a busca no espaço de configurações ocorre em uma fase de treinamento e a melhor configuração encontrada é, então, utilizada em produção. Usualmente, utilizamos um conjunto de instâncias de treinamento e, como forma de validar as configurações obtidas, utilizamos um conjunto diferente de instâncias para testar essas configurações obtidas [Birattari & Kacprzyk, 2009].

Definimos aqui o problema de configuração de algoritmos. Seja $\mathcal{A}$ um algoritmo com n parâmetros p_i e $1 \leq i \leq n$, de forma que cada parâmetro p_i possua domínio Θ_i . Temos que o espaço de configurações $\Theta \subseteq \Theta_1 \times \Theta_2 \times \dots \times \Theta_n$ contém todas as combinações de parâmetros válidas do algoritmo $\mathcal{A}$. Dessa maneira, dado um conjunto de instâncias Π e uma função de custo $c(\theta, \pi)$ que mede o custo de executar $\mathcal{A}$ sobre a instância $\pi \in \Pi$ com uma configuração $\theta \in \Theta$, o objetivo do problema é encontrar uma configuração $\theta^* \in \Theta$ que minimize o custo de $\mathcal{A}$ sobre todo o conjunto Π . Podemos ver essa função objetivo na Equação 2.9, onde $\mathcal{C}$ é uma função agregada de $c(\theta, \pi)$ sobre todas as instâncias $\pi \in \Pi$ [De Souza, 2022].

$$\theta^* \in \operatorname{argmin} \mathcal{C}(\theta, \Pi) \quad \theta \in \Theta \quad (2.9)$$

Podemos utilizar como função agregada $\mathcal{C}$, a exemplo, o desempenho médio em todas as instâncias $\mathcal{C}(\theta, \Pi) = \sum_{\pi \in \Pi} c(\theta, \pi) / |\Pi|$ ou o pior desempenho encontrado em qualquer uma das instâncias $\mathcal{C}(\theta, \Pi) = \max_{\pi \in \Pi} c(\theta, \pi)$.

Um algoritmo parametrizado pode ter parâmetros de diferentes tipos. Nesse sentido, parâmetros categóricos definem um conjunto fixo de opções para o algoritmo. Utilizamos esse tipo de parâmetro frequentemente para definir estratégias do algoritmo para atingir um mesmo fim, por exemplo, qual vizinhança utilizar em uma busca local. Os parâmetros também podem ser ordinais, isto é, seus valores possuem uma relação de ordem intrínseca. Por exemplo, considere o mesmo parâmetro de escolha de vizinhança em uma busca local, mas, agora, as vizinhanças estão ordenadas pelo seu tamanho. Por fim, também temos os parâmetros numéricos. Eles mapeiam valores inteiros ou reais do algoritmo, como, por exemplo, constantes utilizadas em cálculos e funções, ou a temperatura inicial de um algoritmo de Recozimento Simulado (do inglês, *Simulated Annealing* - [SA](#)) [López-Ibáñez et al., 2016].

Além dos diferentes tipos, podemos definir relações entre parâmetros. Uma relação condicional acontece quando um parâmetro pode ser definido apenas se outro parâmetro também for definido. Por exemplo, em um algoritmo que utiliza diferentes meta-heurísticas definidas por um parâmetro, só faz sentido definir o parâmetro *tenure* se o algoritmo alvo utilizar a meta-heurística Busca Tabu. Parâmetros também podem impor restrições a outros parâmetros. Por exemplo, em um algoritmo evolutivo, o número de indivíduos selecionados para o processo de *crossover* não pode ser maior do que o tamanho da população. Por fim, algumas combinações de parâmetros não são permitidas, pois podem causar erros ou comportamentos indefinidos no algoritmo [López-Ibáñez et al., 2016].

Assim, podemos definir uma instância do problema de configuração de algoritmos. Ela é uma sêxtupla $I = (\mathcal{A}, \Theta, \Pi, c, B, S)$ onde $\mathcal{A}$ é o algoritmo alvo da configuração, Θ é o espaço de configurações, Π é um conjunto de instâncias de treinamento, c é uma métrica de custo, B é a quantidade de recursos disponíveis para configuração e S é um conjunto de informações adicionais [Hutter et al., 2009]. Mais especificamente, o recurso B diz respeito ao recurso computacional disponível para realizar essa configuração e, usualmente, medimos em iterações ou em tempo de processamento. Além disso, também temos as configurações adicionais S que incluem, por exemplo, fatores relacionados ao processo de configuração, a alguma configuração do algoritmo ou às instâncias de teste. Por exemplo, o conjunto S pode conter configurações iniciais para o configurador, o critério de parada do algoritmo ou argumentos específicos para certas instâncias.

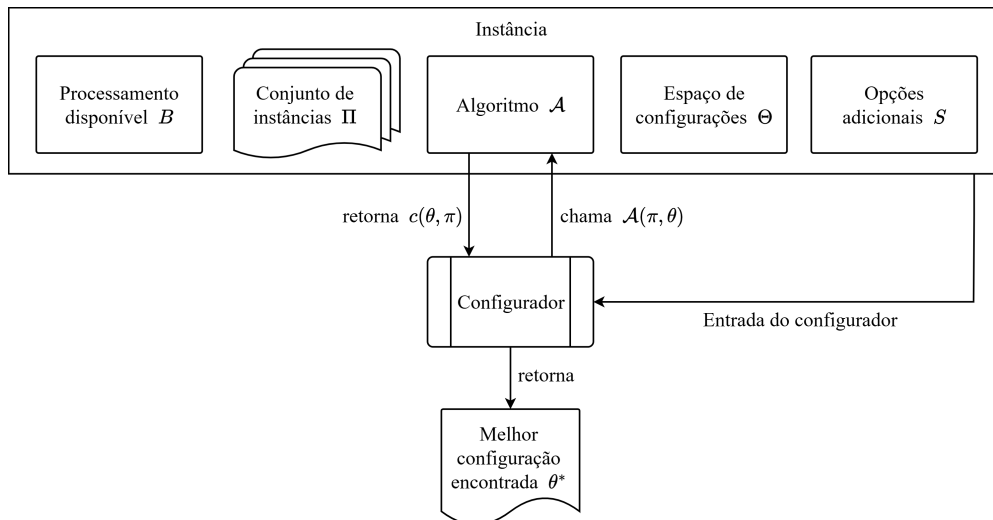
Mesmo com o problema formalizado, precisamos de maneiras de resolvê-lo. Uma

abordagem possível seria fazer isso manualmente através de testes e observação das melhores configurações. Porém, como já mencionamos, esse tipo de processo ocupa muito tempo dos projetistas de algoritmos, principalmente para testar configurações ruins. Além disso, o processo é muito difícil de replicar e pode gerar configurações enviesadas que, conseqüentemente, produzem resultados inválidos. Por esses motivos, precisamos de métodos sistemáticos para resolver o problema da configuração de algoritmos [Birattari & Kacprzyk, 2009]

2.3.4 Processo de configuração automática e configuradores

O processo de AAC consiste em utilizar os recursos computacionais disponíveis para explorar o espaço de configurações. Chamamos a ferramenta que realiza essa exploração de configurador. Na Figura 2.7 podemos observar a organização e o fluxo dos componentes de um configurador. O algoritmo configurador recebe como entrada a instância definida na subseção anterior. A partir disso, ele realiza chamadas do algoritmo $\mathcal{A}$ sobre uma determinada instância π com a configuração θ . O algoritmo $\mathcal{A}$, então, retorna o valor da métrica c para a instância e configuração fornecidas. Dessa maneira, o configurador avalia configurações e explora o espaço de busca sistematicamente. Ao consumir todo o recurso computacional B , o configurador retorna a configuração (ou configurações) que teve o melhor desempenho agregado nas instâncias de treinamento. [Hutter et al., 2009].

Figura 2.7. Fluxo de execução de um algoritmo configurador



Fonte: Adaptado de De Souza [2022]

O processo de AAC é desafiador por diversos motivos. O trabalho de De Souza [2022] organiza esses desafios, identificados durante o desenvolvimento dos diversos

configuradores da literatura. Um dos desafios a superar é a dificuldade de obter um conjunto de configurações que otimize a função objetivo para todas as instâncias de treinamento. O configurador precisa avaliar corretamente as métricas de qualidade independentemente da instância para que ele explore corretamente o espaço de soluções. Por exemplo, instâncias com custos de escalas diferentes não podem enviesar o configurador, fazendo-o focar apenas na otimização da instância de maior magnitude.

Outro desafio é a natureza estocástica de certos algoritmos. Nesses casos, podemos precisar de múltiplas execuções do algoritmo alvo para obter uma informação relevante sobre a qualidade de uma configuração. A flexibilidade dos parâmetros também gera dificuldades para o configurador, pois causa interações entre os parâmetros em diferentes magnitudes. O configurador precisa identificar tais interações e utilizar essa informação no processo de busca.

A solução mais óbvia para alguns desses desafios seria realizar mais execuções do algoritmo alvo para obter mais informações. Porém, realizar uma execução é um processo custoso e, devido à limitação de tempo e ao poder de processamento, não devemos desperdiçar execuções. Desta forma, o configurador precisa implementar estratégias que superem esses desafios, minimizando o número de execuções do algoritmo alvo.

Além desses pontos que dificultam a exploração eficiente do espaço de configuração de um algoritmo, o usuário de um configurador também deve levar em conta alguns elementos para extrair todo o potencial dos métodos de AAC. Em relação às instâncias que utilizamos no processo, é essencial que façamos uma divisão entre as instâncias de treinamento e teste, pois isso nos permite realizar uma avaliação precisa da efetividade da configuração. Aliado a isso, as instâncias que selecionarmos para o treinamento devem representar as diversas características do restante das instâncias. Essa representatividade garante que o configurador tenha contato com as propriedades das instâncias do problema e, conseqüentemente, encontre configurações que funcionem bem para as instâncias de teste ou produção [Eggenberger et al., 2019].

Ainda em relação às instâncias, muitas instâncias difíceis no conjunto de treino farão com que o configurador consuma rapidamente os recursos disponíveis, visto que a maior parte do tempo será gasta pelo algoritmo alvo resolvendo essas instâncias difíceis. Dessa forma, precisamos equilibrar a quantidade de instâncias difíceis no conjunto de treinamento. Esse equilíbrio é crucial, pois, se utilizarmos um conjunto de treino fácil demais, o configurador pode ficar enviesado para instâncias fáceis [Eggenberger et al., 2019].

O recurso computacional que disponibilizamos ao configurador deve ser coerente com o cenário de configuração. Quanto maior o espaço de configuração, maior deve ser a quantidade de recurso computacional fornecida ao configurador para uma exploração

completa do espaço de busca [Eggensperger et al., 2019]. Nesse sentido, o trabalho de Hutter et al. [2017] comprova essa afirmação. Ademais, os autores também concluem que, apesar de os configuradores precisarem de poucos recursos para encontrar boas configurações em algoritmos com um espaço de configuração pequeno, dado o tempo e o recurso computacional necessários, a configuração de um algoritmo que possui um grande espaço de configuração gera um ganho relativamente maior.

Ainda no âmbito do espaço de configuração, precisamos escolher bem quais parâmetros de um algoritmo vamos configurar automaticamente e quais serão seus intervalos, pois a adição de poucos parâmetros pode aumentar drasticamente o espaço de configuração. Apesar de a configuração de uma grande quantidade de parâmetros poder gerar uma configuração mais refinada, precisamos considerar o aumento da dificuldade do processo de configuração. Nessa perspectiva, realizar uma configuração de parâmetros cuidadosamente selecionados pode trazer resultados melhores em menos tempo do que tentar configurar todos os parâmetros do algoritmo [Eggensperger et al., 2019].

2.4 Projeto automático de algoritmos

Até o momento, vimos as características da AAC. Explicitamos as motivações para utilizar a AAC e os benefícios que essa utilização oferece. Em resumo, essas técnicas nos fornecem uma maneira científica e automatizada de encontrar combinações otimizadas de parâmetros para um algoritmo. Essa automatização auxilia os projetistas de algoritmos na medida em que oferece informações realistas e precisas sobre a efetividade dos componentes que eles desenvolveram. Porém, nesse cenário, a combinação de tais componentes ainda é responsabilidade do projetista. Com isso em mente, uma abordagem mais sofisticada e ambiciosa seria expormos não só os parâmetros do algoritmo para configuração automática, mas também a combinação de diferentes componentes algorítmicos. Logo, todo o processo mecânico do projeto de algoritmos seria realizado de forma automática [Stützle & López-Ibáñez, 2018]. Nesta seção, apresentaremos os conceitos fundamentais do AAD.

Primeiramente, demonstraremos a complexidade e as nuances envolvidas no projeto de meta-heurísticas como forma de ilustrar as vantagens no AAD. Dessa forma, considere a ILS representada no Algoritmo 1 como exemplo (por simplicidade, o algoritmo não trata explicitamente o armazenamento da melhor solução encontrada, mas assumimos que esse processo ocorre). A ILS possui quatro componentes principais: solução inicial, perturbação, melhora e critério de aceitação. Cada uma dessas componentes suporta diversas implementações internas específicas que o projetista deve fazer para obter um algoritmo válido [Lourenço et al., 2018].

Algoritmo 1 Busca Local Iterada**Saída:** s : Melhor solução encontrada durante o processo

```

1:  $s_0 \leftarrow \text{GeraSolucaoInicial}()$ 
2:  $s^* \leftarrow \text{Melhora}(s_0)$ 
3: enquanto Critério de parada não for atingido faça
4:    $s' \leftarrow \text{Perturba}(s^*)$ 
5:    $s'' \leftarrow \text{Melhora}(s')$ 
6:    $s^* \leftarrow \text{Aceita}(s^*, s'')$ 
7: fim enquanto

```

Podemos implementar o método `GeraSolucaoInicial` como, por exemplo, uma heurística construtiva gulosa ou como uma geração aleatória de soluções. Cada uma dessas estratégias possui suas peculiaridades, mas não as abordaremos aqui. Veja Talbi [2009] para uma visão geral das técnicas e Lourenço et al. [2018] para um detalhamento das estratégias dentro de uma [ILS](#).

Podemos implementar o método `Perturba` de várias formas. Uma das formas é utilizar as diferentes vizinhanças do problema para modificar soluções. Outra forma é realizar modificações aleatórias em soluções. Esses métodos podem ser combinados de qualquer maneira para gerar perturbações mais sofisticadas, além de que o projetista pode fixar essas combinações ou decidir realizá-las dinamicamente durante a execução. Outro fator é a força das modificações realizadas, isto é, o quão diferentes serão as soluções resultantes. Esse fator pode ser fixo, incrementado quando a busca fica presa em ótimos locais ou o algoritmo pode determiná-lo dinamicamente [Stützle & López-Ibáñez, 2018; Lourenço et al., 2018].

Dada uma solução, a tarefa do procedimento `Melhora` é encontrar um ótimo local e, por isso, ele também é altamente personalizável. Desta forma, podemos utilizar qualquer heurística de busca de solução única para esse fim, incluindo outro procedimento [ILS](#) [Talbi, 2009]. Ademais, cada um dos possíveis procedimentos de busca adiciona uma série de decisões de projeto internas, como vizinhanças, regras de comportamento ou simplesmente novos parâmetros numéricos [Stützle & López-Ibáñez, 2018].

Por fim, o procedimento `Aceita` varia em um espectro de aceitar apenas soluções que melhoram a função objetivo ou aceitar qualquer nova solução. As opções intermediárias nesse espectro incluem, por exemplo, utilizar modelos probabilísticos como o critério de Metrópolis, que sempre aceita soluções melhores, mas também aceita soluções piores por uma certa probabilidade $\exp((f(s^*) - f(s''))/T)$ onde T é um parâmetro chamado temperatura. Esse parâmetro pode ser fixo ou variável. No segundo caso, semelhante à meta-heurística Recozimento Simulado (do inglês, *Simulated Annealing*) ([SA](#)), o projetista deve definir também o esquema de variação da temperatura [Lourenço et al., 2018]. Para mais detalhes do critério de Metrópolis e [SA](#) veja Delahaye

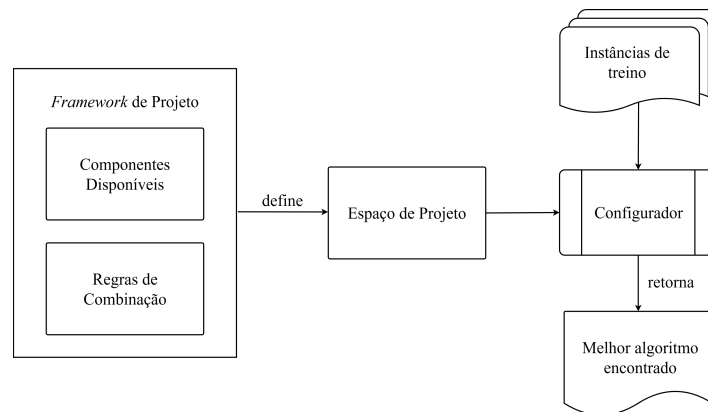
et al. [2018] e Talbi [2009].

A partir dessa listagem de possibilidades de implementação, concluímos que o processo de implementação de um algoritmo ILS efetivo exige que o projetista realize uma exploração manual das diferentes alternativas por meio de testes, intuições sobre o problema-alvo e experiências prévias de outras implementações. Esse trabalho manual possui desvantagens muito parecidas com as desvantagens da configuração manual de parâmetros, ou seja, é um processo dificilmente reproduzível, tedioso, custoso e tende a gerar algoritmos que não utilizam todo o potencial das componentes disponíveis [Stützle & López-Ibáñez, 2018].

Dessa forma, com o objetivo de mitigar esses problemas, a comunidade científica expandiu os conceitos de AAC para automatizar também a exploração do espaço de algoritmos para um determinado problema. Para isso, os componentes algorítmicos, as regras de combinação desses componentes e os parâmetros do algoritmo formam o espaço de configuração. A partir disso, podemos utilizar as técnicas de AAC já existentes na exploração deste espaço.

Com isso estabelecido, a responsabilidade do projetista é utilizar sua intuição e conhecimento do problema para desenvolver componentes eficientes. Conforme representado na Figura 2.8, dados esses componentes e as regras para combiná-los, os configuradores podem realizar o processo de combinação automaticamente.

Figura 2.8. Fluxo de um configurador no processo de AAD



Fonte: Adaptado de De Souza [2022]

2.4.1 Abordagem *Top-Down*

Uma abordagem *Top-Down*, de forma geral, significa observar o problema do mais alto nível para o mais baixo. Mais especificamente, no contexto do projeto automático de algoritmos, isso significa determinar um modelo de alto nível de um algoritmo e utilizar componentes específicos para instanciar os componentes genéricos desse modelo.

Nessa abordagem, o algoritmo resultante sempre terá o formato do modelo definido [Mascia et al., 2013]. Por exemplo, podemos considerar o Algoritmo 1 explicado anteriormente como nosso modelo. Recapitulando, precisamos definir quatro componentes para obter um algoritmo de ILS: solução inicial, melhoria, perturbação e critério de aceitação. Dessa forma, independentemente de quais componentes específicos escolhermos instanciar, ao final do processo, sempre obteremos uma ILS. A Tabela 2.2 reúne possibilidades de implementação para cada um dos componentes genéricos do modelo.

Tabela 2.2. Estratégias disponíveis para cada componente da meta-heurística ILS

Componente	Estratégias
Solução Inicial	Construtiva, Aleatória
Melhoria	Busca Tabu, Busca Local, Recozimento Simulado
Perturbação	Vizinho Aleatório, Alteração Aleatória
Critério de Aceitação	Aceita Melhora, Aceita Qualquer, Metrópolis

Fonte: Os Autores.

A partir da definição dessas componentes específicas, podemos instanciar diferentes algoritmos de ILS a partir do modelo. Dessa maneira, podemos buscar as combinações de componentes mais eficientes. No processo de definição dos componentes, precisamos levar em conta que o número de componentes específicas disponíveis afeta diretamente a quantidade de algoritmos que podemos gerar. Por isso, precisamos equilibrar a variabilidade de opções com a complexidade de explorar esse espaço de configuração [De Souza, 2022].

Nesse tipo de abordagem, podemos modelar as regras de combinação dos componentes como parâmetros categóricos de um algoritmo. Por exemplo, poderíamos implementar cada um dos métodos de melhoria e definir um parâmetro categórico p_i que determinaria qual estratégia de melhoria o algoritmo utilizaria na execução de forma que $p_i \in \{\text{buscaTabu}, \text{buscaLocal}, \text{recozimentoSimulado}\}$. Além disso, certas estratégias precisam de parâmetros adicionais; por exemplo, ao instanciarmos a busca tabu como processo de busca, precisamos definir qual vizinhança utilizar e um valor para o parâmetro *tenure*. Podemos modelar esses parâmetros adicionais como parâmetros numéricos ou categóricos condicionais. Dessa forma, a partir dessas ideias, podemos parametrizar as decisões de projeto de algoritmos e utilizar os configuradores já existentes para gerar algoritmos automaticamente. [De Souza, 2022]

Através de trabalhos como KhudaBukhsh et al. [2016], Franzin & Stützle [2019] e Bezerra et al. [2020], podemos verificar a efetividade da abordagem *Top-Down* em gerar algoritmos efetivos (veja a Seção 3.2 para mais detalhes sobre os trabalhos da literatura de AAC e AAD). Porém, para utilizá-la eficientemente, precisamos determinar previamente o modelo do algoritmo e considerar as possíveis interações entre

cada um dos componentes implementados. Essa tarefa se torna mais difícil à medida que o número de componentes cresce [Mascia et al., 2014]. Por esse motivo, podemos utilizar, alternativamente, a estratégia *Bottom-Up* para automatizar o processo de desenvolvimento de algoritmos de forma mais flexível.

2.4.2 Abordagem *Bottom-Up*

A abordagem *Bottom-Up* nos permite focar no desenvolvimento dos componentes algorítmicos sem precisarmos determinar o formato do algoritmo final. Além de simplificar algumas fases do processo, essa abordagem tende a gerar algoritmos totalmente novos e híbridos.

Apesar da flexibilidade na geração de algoritmos, ainda precisamos definir as regras de combinação dos componentes. Uma das formas de representar esse cenário é utilizando uma gramática livre de contexto na forma Backus-Naur. Definimos essa gramática de forma que qualquer derivação das regras gere um algoritmo válido [Mascia et al., 2013]. Na Figura 2.9 podemos ver um exemplo de gramática capaz de gerar algoritmos compostos por diferentes combinações de componentes. Para a definição e teoria em gramáticas livres de contexto, veja Lewis & Papadimitriou [1997].

Figura 2.9. Gramática livre de contexto na forma Backus-Naur para geração de um algoritmo de busca

```

<alg> ::= <solIni>
        <busca>
<solIni> ::= constr() | rand()
<busca> ::= bt(<tenure>, <melhora>) | bl(<melhora>) |
          sa(<temp>) | ils(<busca>, <pert>)
<melhora> ::= bi | fi
<tenure> ::= [5,500]
<pert> ::= rand1(<tam>) | rand2(<tam>)
<tam> ::= [1, 100]
<temp> ::= 500 | 1000 | 5000 | 10000

```

Fonte: Os Autores.

A gramática é composta por uma série de regras de transformação de símbolos não terminais. A forma Backus-Naur representa os símbolos não terminais entre os caracteres <>. Cada regra define uma escolha de projeto. Por exemplo, o símbolo inicial da gramática de exemplo é o <alg>. Esse símbolo deriva para os não terminais de solução inicial <solIni> e processo de busca <busca>, indicando que esses componentes devem existir no algoritmo. O primeiro símbolo deriva para algumas estratégias de gerar soluções iniciais, enquanto o segundo tem o potencial de gerar métodos de busca

inteiros sofisticados e híbridos. Cada um dos métodos de busca **bt** (Busca Tabu), **bl** (Busca Local), **sa** (*Simulated Annealing*) e **ils** (Busca Local Iterada) possui seus próprios argumentos, os quais outros símbolos não terminais determinam; inclusive, um deles recebe outro método de busca como argumento, gerando recursividade na gramática.

Podemos notar que os símbolos não terminais da gramática derivam ou para um conjunto determinado de valores ou para um intervalo numérico específico. A partir dessa percepção, Mascia et al. [2013] propôs mapear os símbolos não terminais para parâmetros categóricos ou numéricos em um algoritmo e, com isso, utilizar técnicas de [AAC](#) para realizar a derivação da gramática. Dessa forma, um símbolo não terminal que deriva de outro símbolo não terminal se torna um parâmetro condicional em relação ao parâmetro do símbolo gerador. Também precisamos tratar da recursividade da gramática, visto que um algoritmo possui um conjunto finito de parâmetros. Mascia et al. [2013] propõe criar cópias dos parâmetros para cada nível permitido de recursão, de forma que os parâmetros do nível $i + 1$ sejam condicionais ao parâmetro que gera a recursão no nível i .

Capítulo 3

Trabalhos Relacionados

O [JITJSS](#) foi proposto pela primeira vez em 2008 por Baptiste et al. [2008]. Desde então, diferentes autores estudaram o problema e propuseram diferentes estratégias para resolvê-lo. Nesta seção, listaremos com detalhes os trabalhos que abordaram o problema. Além disso, ainda não há nenhum trabalho que aplicou as técnicas de [AAD](#) especificamente ao [JITJSS](#). Por isso, como forma de justificar este trabalho, apresentaremos os usos de [AAC](#) e [AAD](#) da literatura para encontrar algoritmos estado da arte para diversos problemas.

3.1 *Just-In-Time Job Shop*

Baptiste et al. [2008] propuseram as 72 instâncias do problema que foram utilizadas como métrica de qualidade em todos os outros trabalhos que estudaram o [JITJSS](#). Os autores também desenvolveram duas relaxações lagrangianas para encontrar limites inferiores para cada instância. Essas estratégias relaxam as restrições de precedência das tarefas e as restrições de máquina. A partir desses limites inferiores, eles utilizam métodos de *branch-and-bound* e programação linear para encontrar soluções viáveis e uma busca local para melhorar as soluções encontradas. Nesse sentido, o trabalho propôs os primeiros limites superiores para as instâncias, mas, em diversas instâncias, a diferença entre os limites inferiores e superiores ainda era grande.

Laborie & Godard [2007] desenvolveram um método genérico para resolver problemas de agendamento que utiliza uma busca auto-adaptativa em grandes vizinhanças, baseada em relaxar soluções através de um portfólio de vizinhanças e reotimizá-las. Com essa abordagem, eles conseguiram limites superiores melhores em diversas instâncias. Monette et al. [2009] também utilizaram um processo de relaxação das máquinas, porém, aplicado em um algoritmo *branch-and-bound* desenvolvido por eles. Os autores também acoplaram dois métodos de busca local no processo e, com isso, conseguiram

melhorar o estado da arte do problema.

Outro tipo de abordagem utilizada são os algoritmos evolucionários. Araujo et al. [2009] propuseram um algoritmo genético combinado com uma busca local que foi expandido por Dos Santos et al. [2010]. No segundo trabalho, os autores utilizam um método de programação linear para gerar um agendamento através de uma sequência. Isso torna o modelo menos flexível e mais contido, gerando agendamentos ótimos em menos tempo. Com essas abordagens, os autores melhoraram, na época, 56 das 72 instâncias do estado da arte.

Ainda no âmbito de algoritmos evolucionários, Yang et al. [2012] propõem em seu trabalho um algoritmo genético que difere dos anteriores, de forma mais significativa, na decodificação de um cromossomo. Os autores desenvolveram um método dividido em três estágios para gerar agendamentos válidos. Esse método obteve resultados ligeiramente melhores em algumas instâncias de 10 e 15 tarefas, e os autores não reportaram os resultados para as instâncias maiores.

A meta-heurística [VNS](#) também já foi aplicada ao problema. Em Wang & Li [2014], os autores propõem um [VNS](#) com duas vizinhanças. Além disso, eles utilizam a mesma estratégia utilizada por Dos Santos et al. [2010] na geração de agendamentos, isto é, utilizam uma meta-heurística para explorar o espaço de sequências e um método de programação inteira para gerar um agendamento ótimo para cada sequência. Os autores conseguiram encontrar vários resultados melhores que o estado da arte, principalmente nas instâncias de 10 e 15 tarefas.

Ahmadian et al. [2021] também utilizaram um [VNS](#) para resolver o problema. Porém, nesse trabalho, além de eles utilizarem programação inteira para gerar agendamentos a partir de sequências, eles utilizam esses métodos para melhorar as próprias sequências. Para isso, os autores propuseram selecionar um subconjunto das tarefas de uma máquina e relaxar as restrições de precedência dessas operações. Por conseguinte, o modelo passa a procurar a ordem ótima apenas para esse subconjunto de operações selecionadas. Essa estratégia simplifica o modelo de programação inteira, gerando resultados melhores em menos tempo. Essa proposta conseguiu atingir ou melhorar o estado da arte em 61 das 72 instâncias, tornando-se a solução mais eficiente e abrangente da literatura.

Mais recentemente, Abderrazzak et al. [2022] propuseram novamente uma busca em grandes vizinhanças para resolver o problema. Essa abordagem, porém, consiste em destruir e reconstruir as soluções de forma adaptativa durante a execução do algoritmo. Os autores conseguiram atingir o estado da arte na maior parte das instâncias de 10 e 15 tarefas e melhorar algumas instâncias pontualmente. Além dessa estratégia, Sabri et al. [2023] recentemente propuseram uma abordagem ao problema com aprendizado por

reforço profundo que obteve poucas melhorias na qualidade das soluções, mas obteve soluções competitivas para as instâncias de 10 e 15 tarefas utilizando pouco tempo de processamento em comparação com o restante da literatura. Ambos os trabalhos não reportaram os resultados para as maiores instâncias do problema.

Ao longo dos anos, esses trabalhos encontraram soluções cada vez melhores para as instâncias propostas por Baptiste et al. [2008]. Nesse sentido, as melhores soluções encontradas para cada instância estão contidas em cinco trabalhos: Dos Santos et al. [2010]; Wang & Li [2014]; Ahmadian et al. [2021]; Abderrazzak et al. [2022]; Sabri et al. [2023]. Dessa forma, quando nos referirmos ao estado da arte para o problema [JITJS](#), estaremos referenciando a composição dos resultados desses cinco trabalhos.

3.2 Projeto Automático de Algoritmos

Em Liao et al. [2014], os autores desenvolveram um Otimização por Colônia de Formigas (do inglês, *Ant Colony Optimazation* – [ACO](#)) para otimização contínua que combina componentes de outros métodos Otimização por Colônia de Formigas (do inglês, *Ant Colony Optimazation*) ([ACO](#)) da literatura em um ambiente genérico de configuração. Dessa forma, o algoritmo desenvolvido pode instanciar tanto as estratégias da literatura quanto combinações de componentes nunca vistas. Os autores utilizaram configuração automática para instanciar os algoritmos e, com isso, eles obtiveram resultados melhores em comparação a outros trabalhos da literatura.

Os trabalhos de Aydın et al. [2017] e Bezerra et al. [2020] realizam um processo parecido no sentido de reunir estratégias verificadas da literatura para uma meta-heurística específica, mas eles abordam, respectivamente, os algoritmos de colônia artificial de abelhas e algoritmos evolucionários multiobjetivos. Em ambos os trabalhos, os autores reportaram que a performance dos algoritmos gerados era maior quando comparada a outros algoritmos da literatura.

Com uma abordagem menos centrada no tipo de meta-heurística utilizada, KhudaBukhsh et al. [2016] utilizam uma abordagem *top-down* de projeto automático de algoritmos para gerar buscas locais estocásticas que resolvem o problema da Problema da Satisfabilidade Proposicional (do inglês, *Boolean Satisfiability Problem* ([SAT](#))). Eles comparam os algoritmos gerados pela sua abordagem com outros 11 *solvers* do [SAT](#) e reportam resultados melhores do que os obtidos pelos outros *solvers*.

Já De Souza & Ritt [2018a] utilizam uma abordagem *bottom-up* para gerar heurísticas automaticamente para a [UBQP](#). Os autores reportam resultados competitivos com estado da arte obtidos pelos algoritmos gerados. Em De Souza & Ritt [2018b], os autores apresentam uma redução do problema de alocação de provas para [UBQP](#)

e utilizam novamente uma abordagem *bottom-up* para gerar heurísticas para o problema. A abordagem proposta obteve resultados melhores que o estado da arte para o problema de alocação de provas, mostrando-se uma alternativa para resolver também outros problemas redutíveis ao [UBQP](#).

No âmbito de problemas de agendamento, as variações do [FSP](#) foram muito abordadas com técnicas de projeto automático. Marmion et al. [2013] propõem a estrutura de uma busca local estocástica híbrida genérica para gerar algoritmos para o problema *Permutation Flow Shop Problem* com atraso ponderado ([PFSP-WT](#)). Os autores geraram três algoritmos diferentes. Com isso, eles compararam os resultados com o algoritmo estado da arte do problema e, para cada instância, pelo menos um dos algoritmos era melhor que o estado da arte, sendo que, na maioria das instâncias, os três algoritmos gerados eram melhores que o estado da arte.

Pagnozzi & Stützle [2019] também propuseram um espaço de projeto automático de buscas locais estocásticas híbridas. Nesse trabalho, os autores abordaram três funções objetivo diferentes para o [PFSP](#). Em comparação com o estado da arte para cada tipo de função objetivo, dois dos três algoritmos gerados se tornaram o novo estado da arte para as suas respectivas funções objetivo.

Brum & Ritt [2018] também abordam o [PFSP](#) através do projeto automático de algoritmos. Porém, neste trabalho, os autores desenvolvem um modelo de geração de algoritmos baseado na meta-heurística [ILS](#). Os autores geraram 10 algoritmos e compararam a performance deles contra o estado da arte. Dessa maneira, eles obtiveram resultados melhores que o estado da arte com a maioria dos algoritmos gerados.

Alfaro-Fernández et al. [2020] propõem soluções para três funções objetivo do problema [HFS](#). Eles também utilizam um modelo de [ILS](#) para gerar algoritmos automaticamente. Os autores conseguiram gerar dois novos algoritmos estado da arte para as funções objetivo *Total Flow Time* ([TFT](#)) e Atraso e Adiantamento Totais Ponderados.

Brum et al. [2022] aborda outro problema chamado [NPFSP](#). Os autores dividem o processo em duas fases. A primeira consiste em resolver a instância com as restrições do [PFSP](#). A segunda fase consiste em resolver o [NPFSP](#) utilizando a solução da primeira fase como solução inicial. A partir dessa ideia, os autores geram automaticamente algoritmos para o [NPFSP](#). Os algoritmos gerados obtiveram melhores resultados que o estado da arte tanto para o [PFSP](#) quanto para o [NPFSP](#); isto é, os resultados gerados pela primeira fase já eram melhores que o estado da arte para o [PFSP](#) e, ao receber essas soluções como entrada, a segunda fase também obteve resultados melhores que o estado da arte para o [NPFSP](#).

Todos esses trabalhos têm como característica comum obter algoritmos que superaram total ou parcialmente o estado da arte em seus respectivos domínios com as

técnicas de AAC e AAD. Além disso, apesar de nenhum trabalho ter aplicado essas técnicas ao JITJSS, temos diversos trabalhos que demonstraram a eficácia da AAC e do AAD em outros problemas de agendamento. A partir disso, o presente trabalho se sustenta na hipótese de que a aplicação das técnicas de AAC e AAD ao JITJSS resultará em novos algoritmos estado da arte para o problema.

Capítulo 4

Metodologia

Para atingir os objetivos especificados no Capítulo 1, realizamos um processo baseado no fluxo dos trabalhos relacionados de AAD. Primeiramente, realizamos uma seleção de componentes algorítmicos de alto e baixo nível da literatura para compor nosso espaço de projeto. Esses componentes foram prioritariamente selecionados da literatura do JITJSS, mas também consideramos estratégias de outros problemas de agendamento na construção do espaço de projeto. Além disso, propomos um novo componente heurístico para o problema JITJSS, visando enriquecer o espaço de projeto com características específicas do problema que ainda não foram exploradas.

Após definir o espaço de projeto, fizemos a configuração do cenário de projeto para iniciar a construção automática de algoritmos. Para isso, analisamos o espaço de projeto e parametrizamos a geração de algoritmos em um *framework* configurável. Especificamente, implementamos cada um dos componentes do espaço de projeto de maneira que fossem combináveis por meio de parâmetros em um algoritmo.

Definimos também o conjunto de instâncias que foi utilizado no cenário de projeto automático. Nesse sentido, para obter resultados significativos, definimos um conjunto de treino e outro conjunto de teste. Utilizamos as instâncias descritas na Seção 4.1 como instâncias de teste. E utilizamos o processo de geração descrito na Seção 4.1 para gerar um novo conjunto de instâncias de treino.

Com esses elementos estabelecidos, analisamos o cenário de projeto proposto e definimos o recurso computacional disponível para o algoritmo configurador. Com isso, executamos o algoritmo configurador para explorar o espaço de projeto. Ao fim do processo, realizamos a validação dos algoritmos obtidos utilizando o conjunto de instâncias de teste.

Por fim, para validar a qualidade das estratégias encontradas, realizamos a comparação dos resultados com o estado da arte. Essa comparação foi quantificada por meio da diferença percentual entre a melhor solução conhecida e a solução obtida pelas

estratégias encontradas.

4.1 Materiais

Todos os trabalhos que abordaram o [JITJSS](#) utilizam as instâncias propostas por Baptiste et al. [2008] como métrica de qualidade. Por isso, também utilizamos essas instâncias para testar nossas estratégias. Descrevemos aqui como o autor gerou essas instâncias, pois utilizaremos o mesmo procedimento para gerar novas instâncias.

Para cada par (n, m) , onde $n \in \{10, 15, 20\}$ e $m \in \{2, 5, 10\}$, foram geradas 8 instâncias, formando um total de 72 instâncias. Cada instância possui um nome na forma n - m -DD-W-ID, onde DD, W e ID são determinados das seguintes maneiras:

- DD indica o tipo de distância entre datas de entrega de operações consecutivas da mesma tarefa. Caso DD=tight, essa distância é igual ao peso da segunda operação. Caso DD=loose, essa distância é igual ao peso da segunda operação mais um valor inteiro selecionado aleatoriamente no intervalo $[0, 10]$.
- W indica a forma como as penalidades de adiantamento e atraso (α, β) de cada operação são geradas. Caso W=equal, temos que α e β são selecionados aleatoriamente no intervalo $[0, 1; 1]$. Caso W=tard, temos que α é selecionado aleatoriamente no intervalo $[0, 1; 0, 3]$, enquanto β é selecionado aleatoriamente no intervalo $[0, 1; 1]$.
- Foram geradas duas instâncias para cada uma das combinações das outras variáveis. Dessa forma, a variável ID varia entre 1 e 2 e identifica as duas instâncias com a mesma combinação das outras variáveis.

Realizamos todas as implementações na linguagem de programação C++. Além disso, a respeito do processo de configuração, utilizaremos a biblioteca **Irace** [López-Ibáñez et al., 2016] na versão 4.4.3 para explorar nosso espaço de projeto de algoritmos. Todos os processos que utilizarem o solver CPLEX se referem a IBM Corp. [2022] na versão 22.1.1.0. Além disso, toda a geração de números aleatórios foi realizada utilizando a biblioteca PCG [O'Neill, 2014]. Ainda nesse âmbito, os processos de configuração e testes serão realizados em uma máquina com um processador Intel Core i5-11300H, com frequência de 3,1 GHz e 24 GB de memória RAM, executando o sistema operacional Arch Linux.

4.2 Atividades

As seguintes atividades foram realizadas no desenvolvimento deste trabalho:

1. Selecionar componentes algorítmicos da literatura.
2. Implementar componentes em um *framework* configurável.
3. Desenvolver um novo componente para o problema e incrementar o espaço de projeto com esse novo componente.
4. Gerar o conjunto de instâncias de treinamento.
5. Configurar e executar o algoritmo configurador sobre o espaço de projeto.
6. Validar o algoritmo obtido pelo configurador por meio de uma comparação com o estado da arte nas instâncias de teste.
7. Realizar ajustes no cenário de projeto e repetir a Atividade 5.
8. Documentar o algoritmo e os resultados.
9. Finalizar a escrita do trabalho.

4.3 Cronograma

4.3.1 Proposta de cronograma inicial

A Tabela 4.1 exibe a proposta inicial do cronograma das atividades que seria realizada.

Tabela 4.1. Proposta de cronograma inicial

Atividades	Jan.	Fev.	Mar.	Abr.	Mai	Jun.
1	x	x				
2	x	x				
3	x	x				
4		x				
5			x	x		
6			x	x		
7			x	x		
8			x	x	x	
9					x	x

Fonte: Os Autores.

4.3.2 Cronograma executado

A Tabela 4.2 exibe o cronograma das atividades da forma que foram realizadas.

Tabela 4.2. Cronograma de atividades realizado

Atividades	Jan.	Fev.	Mar.	Abr.	Mai	Jun.
1	x	x				
2	x	x				
3	x	x				
4			x			
5			x	x		
6			x	x		
7			x	x		
8			x	x		
9				x	x	

Fonte: Os Autores.

Capítulo 5

Resultados

5.1 Espaço de projeto construído

A fundamentação teórica e o fluxo metodológico estabelecem a construção do espaço de projeto como a etapa inicial para a geração automática de algoritmos. A Tabela 5.1 lista cada um dos componentes selecionados e implementados para compor o espaço de projeto proposto. As subseções seguintes detalham os componentes e suas respectivas implementações.

Tabela 5.1. Componentes estruturais do espaço de projeto de algoritmos.

Categoria	Componente
Solução Inicial	Giffler-Thompson Construtiva via regras de despacho
Regras de Despacho para as Soluções Iniciais	Prazo de entrega mais próximo ALL+CR+SPT Seleção aleatória de operações
Heurísticas de Busca	Busca Local simples Busca Local Iterada Busca Tabu
Vizinhanças	Troca de operações adjacentes Trocas aleatórios Trocas baseados nas penalidades Trocas no caminho crítico das operações Inserções baseadas nas penalidades Inserções aleatórias Relaxamento de Restrições
Agendamento	Agendamento o mais cedo possível Agendamento exato com CPLEX

5.1.1 Soluções iniciais

Implementamos os dois algoritmos construtivos descritos na Seção 2.2.4 como opções de solução inicial. Conforme a fundamentação teórica estabelece, ambos os procedimentos admitem qualquer regra de despacho internamente que selecione uma operação entre uma lista de operações. Nesse sentido, implementamos três regras de despacho como opções para os dois algoritmos.

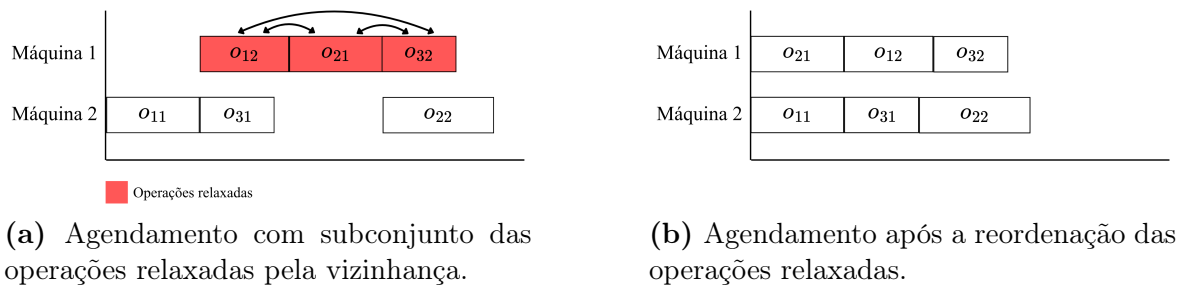
As opções de configuração de regra de despacho compreendem a escolha aleatória de uma operação, a escolha da operação que possuir a Data de Entrega mais Próxima (do inglês, *Earliest Due Date* – **EDD**), e a estratégia híbrida ALL+CR+SPT, definida no trabalho de Wang & Li [2014].

5.1.2 Vizinhanças

No total, o espaço de projeto deste trabalho compreende sete vizinhanças para o problema. A literatura do **JITJSS** aplica e fornece a fundamentação para seis dessas vizinhanças, enquanto a sétima surge como uma proposta original para o problema, fundamentada em conceitos utilizados na resolução do problema **JSS**.

A vizinhança **relax**, baseada em Ahmadian et al. [2021], opera por meio do relaxamento das restrições operacionais. Ela realiza o relaxamento da ordem de um subconjunto de operações e utiliza o solver CPLEX para reorganizar essas operações, gerando, assim, um vizinho da solução original. Essa vizinhança controla a localidade das modificações, mantendo uma parte da solução inalterada. Além disso, o solver CPLEX reorganiza a ordem do subconjunto das operações, otimizando o valor objetivo e, com isso, direcionando a busca para soluções melhores. Contudo, a geração desse vizinho demanda um tempo computacional superior em comparação às demais abordagens, devido à invocação do solver. Por esse motivo, limitamos a execução do solver nessa vizinhança a 1 segundo, como os autores da vizinhança definiram em seu trabalho.

Figura 5.1. Exemplo de modificação realizada pela vizinhança relax.

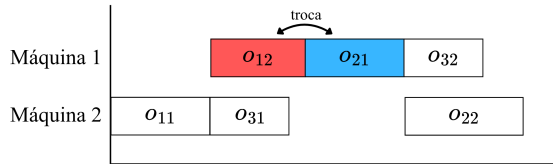


Fonte: adaptado de Ahmadian et al. [2021]

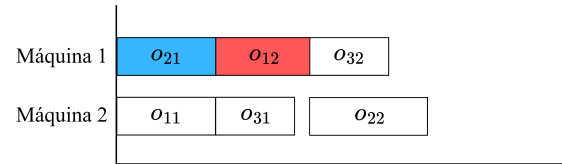
A Figura 5.1 ilustra o processo de otimização realizado por este método. No exemplo da Figura a, o algoritmo relaxa a ordem de três operações, permitindo que o solver CPLEX explore qualquer permutação entre elas. Nesse caso, como representado na figura b, as operações são reorganizadas e, por conta disso, todas as operações de ambas as máquinas são adiantadas.

O projeto divide as demais vizinhanças implementadas em dois grupos: troca e remoção-inserção. As vizinhanças de troca realizam a troca de posição entre duas operações. Já as vizinhanças de remoção-inserção removem uma operação da sua posição e inserem em outra posição da sequência. Recapitulando a Seção 2.2.3, restrições de máquina limitam esses deslocamentos estritamente entre operações processadas na mesma máquina.

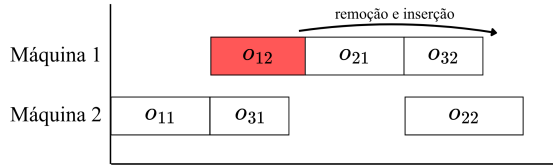
Figura 5.2. Exemplos dos dois tipos principais de vizinhanças: troca e remoção-inserção.



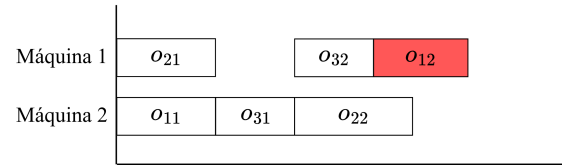
(a) Agendamento antes de acontecer troca entre o_{12} e o_{21}



(b) Agendamento após a troca entre o_{12} e o_{21} .



(c) Agendamento antes de acontecer remoção e inserção de o_{12} .



(d) Agendamento após remoção-inserção com o_{12} no final de uma máquina.

Fonte: os autores.

A Figura 5.2 apresenta exemplos práticos de ambos os tipos de movimentos. As figuras a e b demonstram a execução de uma troca no agendamento. Já as figuras c e d demonstram a execução de uma remoção de uma operação e inserção em outra posição do agendamento.

Primeiramente, temos as vizinhanças aleatórias utilizadas no trabalho de Ahmadian et al. [2021]. Uma delas seleciona duas operações aleatoriamente e realiza a troca de posição entre as duas operações, e a outra remove uma operação aleatória e a insere em outra posição aleatória. Chamaremos a primeira de `troca_al` e a segunda de `insere_al`.

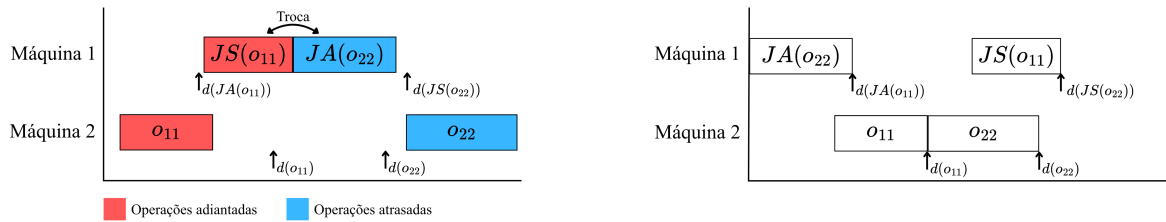
A fundamentação das próximas vizinhanças exige a formalização de definições preliminares. Definimos operações adjacentes entre si como aquelas que não possuem

nenhum tempo ocioso entre elas. Dada uma operação o , a antecessora e a sucessora de tarefa de o são, respectivamente, as operações que ocorrem antes e depois da operação o dentro da ordem da tarefa de o . De forma similar, em uma sequência definida, a antecessora e a sucessora de máquina de o são, respectivamente, as operações que são processadas antes e depois de o na mesma máquina. A notação estabelece $JA(o)$ e $JS(o)$ como a antecessora e sucessora de tarefa, respectivamente, da operação o , que também são adjacentes a o . Da mesma forma, definimos $MA(o)$ e $MS(o)$, respectivamente, como a antecessora e sucessora de máquina da operação o que também são adjacentes a o . Por fim, ao nos referirmos a um bloco de operações, considere um conjunto de operações adjacentes em uma mesma máquina.

A próxima vizinhança possui uma estrutura simplificada. Este componente provém do trabalho de Dos Santos et al. [2010] e realiza a troca de posição entre uma operação e sua sucessora de máquina, sem nenhum tipo de filtro. Partindo de qualquer solução, essa vizinhança gera sempre um número fixo de vizinhos e contém todas as outras vizinhanças que realizam trocas entre operações adjacentes com algum tipo de filtro. A nomenclatura `troca_adj` identifica esta vizinhança.

O trabalho de Wang & Li [2014] fundamenta outras duas vizinhanças que são baseadas nas penalidades das operações. A vizinhança de troca baseia-se no seguinte lema. Se uma operação o está atrasada, $MA(o)$ não existe e $JA(o)$ existe, então $JA(o)$ também está atrasada. E, no caso inverso, se uma operação o está adiantada, $MS(o)$ não existe e $JS(o)$ existe, então $JS(o)$ também está atrasada. Nesse sentido, a partir de uma operação o adiantada (ou atrasada), o algoritmo percorre o agendamento seguindo as operações $JS(o)$ (ou $JA(o)$) até encontrar uma operação que possua uma $MS(o)$ (ou $MA(o)$) ou encontrar uma operação que seja a última (ou a primeira) operação da máquina ou da tarefa. A nomenclatura `troca_penal` identifica esta vizinhança.

Figura 5.3. Exemplo de modificação de soluções pela vizinhança `troca_penal`.



(a) Agendamento com operações atrasadas e adiantadas com possível modificação pela vizinhança `troca_penal`.

(b) Agendamento após a modificação da vizinhança `troca_penal` com todas as operações entregues nos seus prazo de entrega.

Fonte: Adaptado de Wang & Li [2014]

Por exemplo, na Figura 5.3 podemos observar um exemplo de aplicação desta vi-

zinhança. A partir da operação adiantada o_{11} , ao seguir a sucessora $JS(o_{11})$, o procedimento localiza uma troca capaz de zerar as penalidades de todas as operações envolvidas. De forma semelhante, a partir da operação atrasada o_{22} , seguindo a antecessora $JA(o_{22})$, o algoritmo encontraria a mesma troca.

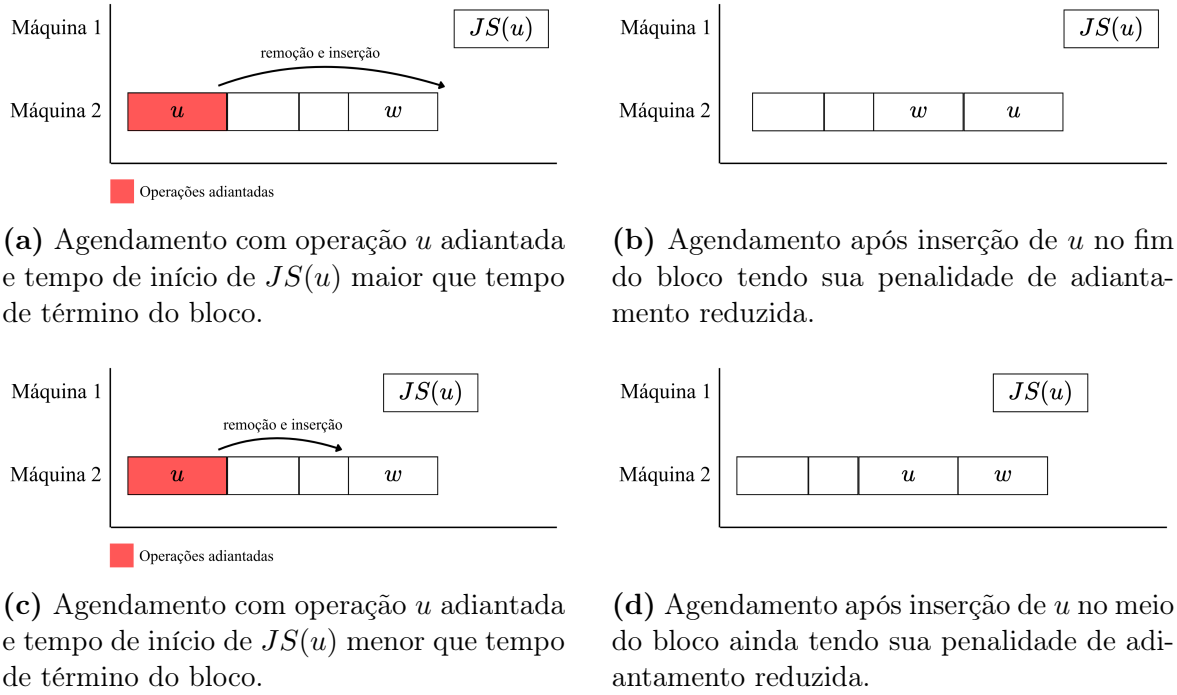
No caso da vizinhança de remoção-inserção baseada nas penalidades, se uma operação está adiantada, a vizinhança remove a operação e tenta inseri-la ao final do bloco correspondente. Por outro lado, se uma operação está atrasada, a vizinhança a remove e tenta inseri-la no início do bloco. Todo o procedimento condiciona a inserção ao agendamento atual para preservar a localidade do movimento. Para uma operação adiantada o que seria inserida ao final do bloco, se o tempo de término do bloco for maior que o tempo de início da sucessora de tarefa de o , então o é inserida após a primeira operação do bloco que possui tempo de término menor que o tempo de início de sua sucessora de tarefa. Da mesma maneira, para uma operação atrasada que seria inserida no início do bloco, se o tempo de início do bloco for menor que o tempo de término da antecessora de tarefa de o , então o será inserida antes da primeira operação do bloco que possuir tempo de início maior que o tempo de término da sua antecessora. A nomenclatura `insere_penal` identifica esta vizinhança.

A Figura 5.4 ilustra como essa modificação acontece, juntamente com o mecanismo de controle do impacto das modificações. Nessa figura, a operação u está adiantada e a vizinhança tentaria inseri-la no fim do seu bloco, isto é, após a operação w . No primeiro cenário, nas figuras a e b, $JS(u)$ tem um tempo de início maior que o término do bloco, ou seja, inserir u no fim do bloco não afetaria o tempo de início de sua sucessora, então, o movimento acontece. Já no segundo cenário, nas figuras c e d, inserir u no fim do bloco não é possível sem atrasar sua sucessora. Por isso, a inserção é realizada antes da operação w , mantendo a posição anterior de $JS(u)$. Quando nossa operação alvo está atrasada, esse processo acontece de forma similar, porém observando os critérios descritos no parágrafo anterior.

Por fim, este trabalho propõe uma vizinhança original ao `JITJSS`, fundamentada na estrutura N5 de Nowicki & Smutnicki [1996] para o problema `JSS`, mas adaptada para a redução de penalidades por atraso. Os autores propõem uma vizinhança que explora o conceito explicado na Seção 2.2.3 de que apenas alterações no caminho crítico de um agendamento alteram seu *makespan*. Nesse sentido, a vizinhança consiste em realizar a troca de posição entre as duas primeiras e as duas últimas operações de cada um dos blocos críticos do agendamento, isto é, blocos de operações que estão contidos no caminho crítico. Perceba que a ideia da otimização do *makespan* é adiantar o tempo de início da última operação do caminho crítico.

Dessa forma, a vizinhança proposta aplica o conceito de caminho crítico individual-

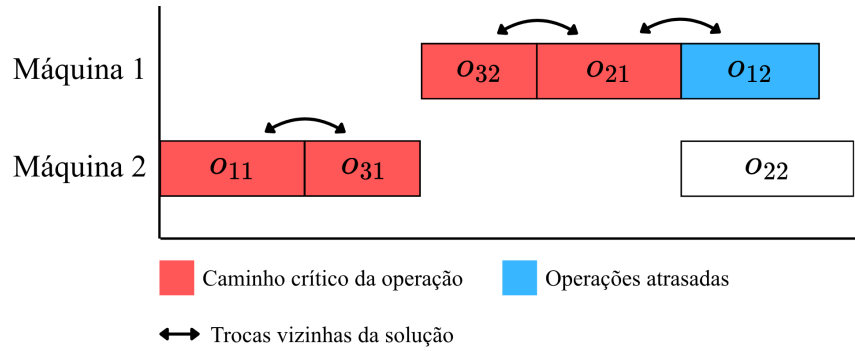
Figura 5.4. Exemplos das modificações da vizinhança `insere_penal` para um operação adiantada.



Fonte: adaptado de Wang & Li [2014].

mente para cada uma das operações atrasadas do agendamento. Para isso, dada uma operação o atrasada, o algoritmo compõe o caminho crítico ao percorrer as antecessoras de o de forma iterativa. A cada iteração, a nova operação a ser adicionada no caminho crítico será a operação com o maior tempo de término entre $JA(o)$ e $MA(o)$, isto é, a operação que impede a operação atual de ser adiantada. A construção do caminho crítico de o termina após encontrar uma operação que não possui nem JA nem MA . A posse desse caminho crítico permite ao algoritmo gerar vizinhos da solução original por meio da estrutura N5. A nomenclatura `troca_crit` identifica esta vizinhança.

A Figura 5.5 apresenta um cenário no qual a operação o_{12} está atrasada, isto é, reduzir seu tempo de início pode contribuir para a redução das penalidades. A aplicação da vizinhança `troca_crit` exige a identificação do caminho crítico específico da operação em questão. A execução do processo descrito no parágrafo anterior resulta na identificação das operações destacadas em vermelho como o caminho crítico de o_{12} . Essas operações representam o caminho de dependência da operação o_{12} , e apenas a modificação delas é capaz de reduzir o tempo de início de o_{12} . A partir disso, esse caminho crítico gera três vizinhos a partir das trocas das operações o_{11} com o_{31} , o_{32} com o_{21} e o_{21} com o_{12} . Trocas estas que correspondem aos pares de operações nas extremidades de cada bloco crítico.

Figura 5.5. Exemplo de modificações vizinhas geradas pela vizinhança troca_crit.

Fonte: Os Autores.

5.1.3 Métodos de busca

A exploração das vizinhanças a partir de uma solução inicial requer o emprego de métodos de busca heurística. Para isso, o projeto contempla três opções de métodos de busca: uma busca local simples conforme explicada na Seção 2.3.2, a busca tabu e a busca local iterada.

A busca tabu segue o que foi definido na seção 2.3.2, mas com a adição de mecanismos que melhoram a intensificação e a exploração do processo de busca. Primeiramente, precisamos definir como a lista tabu armazenará os movimentos que devem ser proibidos enquanto forem tabu. A estrutura de memória registra exclusivamente as arestas do grafo disjuntivo desfeitas pelas modificações na solução. Essa abordagem garante a representação universal de qualquer movimento, uma vez que o grafo disjuntivo sustenta todas as soluções do problema. O detalhe é que cada um dos tipos de modificações é armazenado de forma ligeiramente diferente.

Nas vizinhanças de troca entre duas operações, o algoritmo classifica as arestas de entrada das operações envolvidas como tabu. Por exemplo, se uma máquina possui a seguinte ordem de operações $o1 \rightarrow o2 \rightarrow o3 \rightarrow o4 \rightarrow o5$, então as arestas $\{(o1, o2), (o2, o3), (o3, o4), (o4, o5)\}$ existem no grafo disjuntivo. Se uma vizinhança realiza a troca de posição entre as operações $o2$ e $o4$, então as arestas $(o1, o2)$ e $(o3, o4)$ se tornam tabu. Isso impede, neste exemplo, o retorno das operações $o2$ e $o4$ às posições sucessoras de $o1$ e $o2$ durante a vigência do *tenure*.

Nas vizinhanças de remoção-inserção, apenas a aresta de entrada da operação removida é adicionada como tabu. No mesmo exemplo do parágrafo anterior, se a operação $o2$ for removida e inserida após $o3$, então a aresta $(o1, o2)$ se torna tabu. Com isso, $o2$ é impedida de retornar como sucessora de $o1$ nas próximas iterações.

A verificação da lista tabu acontece durante a exploração da vizinhança pela busca

tabu. Durante esse processo, dada uma lista de opções de modificações geradas por uma vizinhança, o procedimento de busca avalia as modificações geradas e seleciona o movimento conforma a seguinte hierarquia de prioridade:

1. A melhor modificação, seja ela tabu ou não, que seja menor que a melhor solução encontrada até o momento. Este item aplica o critério de aspiração na exploração da vizinhança.
2. A melhor modificação que não seja tabu.
3. A modificação mais antiga da lista tabu.

Todo movimento escolhido é adicionado à lista tabu de acordo com seu tipo, conforme descrito anteriormente. Esse movimento continuará tabu durante um número de iterações determinado pelo *tabu tenure*. A meta-heurística incrementa a contagem de tempo da lista a cada nova inserção. Além disso, caso o algoritmo selecione a modificação tabu mais antiga durante a exploração, o sistema adianta o relógio da lista até o instante em que tal movimento perderia a condição de tabu, removendo simultaneamente da lista elementos mais antigos. Por exemplo, se uma lista tabu possui *tabu tenure* igual a 20 e 5 elementos que foram adicionados há 2, 7, 12, 14 e 16 iterações. Se a modificação com idade 12 foi escolhida dentro de alguma vizinhança, as iterações da lista tabu serão avançadas em 8, fazendo com que os elementos com mais idade que 12 também saiam da lista tabu, restando dois elementos com idades de 10 e 15 iterações.

Além da lista tabu, a implementação integra dois mecanismos baseados no trabalho de Zubaran & Ritt [2016] para otimizar a exploração do espaço de busca: uma lista de soluções elite para intensificação e um detector de ciclos para evitar estagnação. A lista de elites E contém as L melhores soluções encontradas durante a busca. Quando uma nova solução gera melhoria, ela é adicionada a E . Caso E esteja em sua capacidade máxima, a solução mais antiga é removida. Além disso, o sistema armazena, para cada entrada em E , o estado completo da busca, incluindo a lista de vizinhos que ainda não foram explorados e a configuração atual da lista tabu.

A busca inicia em uma solução inicial s^* e explora as vizinhanças a partir dela por um orçamento de até I iterações. O algoritmo reinicia o orçamento de iterações I sempre que identifica uma melhoria na solução atual. Ao atingir o limite de iterações determinado por I , ou ao detectar um ciclo, a busca retoma a exploração a partir da solução elite mais recente, selecionando um de seus vizinhos ainda não explorados. Este movimento de diversificação aplica um fator de redução F sobre o orçamento máximo I . Se nenhuma solução elite existir, a busca para.

Por fim, o detector de ciclos mencionado anteriormente consiste no armazenamento de uma lista de valores de soluções passadas. A partir disso, a busca tabu verifica a repetição desses valores objetivos a cada iteração. Caso essa repetição ocorra mais do que um número máximo de vezes, um ciclo é detectado e o processo descrito no parágrafo anterior é executado.

O terceiro método de busca no espaço de projeto consiste na busca local iterada. Como descrita na Seção 2.3.2, ela é capaz de utilizar internamente tanto a busca local quanto a busca tabu como heurísticas de busca. Ao utilizar a busca local, ela se comporta como uma ILS comum, e ao utilizar a busca tabu, ela se comporta como uma busca tabu iterada. A cada iteração da ILS, ela realiza uma perturbação na solução base. Nesse sentido, o algoritmo utiliza a estratégia de perturbação proposta por Ahmadian et al. [2021]. Ela consiste em escolher duas máquinas aleatoriamente e relaxar as restrições de precedência das máquinas em diversas operações. Dessa forma, o procedimento submete o modelo relaxado ao solver CPLEX com um limite de tempo de 1 segundo, permitindo que a ferramenta reorganize as operações e gere uma nova configuração inicial para a próxima busca local, gerando uma nova solução modificada.

Além disso, considerando que o atual estado da arte para o JITJSS é um VNS, a estrutura da ILS admite a integração de uma coleção de vizinhanças, operando de forma análoga ao VNS. Nesse âmbito, a meta-heurística respeita a ordem de entrada das vizinhanças durante a exploração do espaço de busca. A cada iteração sem melhora, a busca realiza uma perturbação e tenta utilizar a próxima vizinhança da lista em busca de melhorias na solução. Caso aconteça alguma melhora na solução ou caso a coleção de vizinhanças chegue ao fim sem nenhuma melhora, a busca reinicia o ciclo das vizinhanças. Ademais, o espaço de projeto permite a alternância entre os critério de melhor melhora ou primeira melhora para cada vizinhança integrante da lista.

Ambos os métodos, busca local e busca tabu, são capazes de executar a exploração das vizinhanças sob os critérios de melhor melhora ou primeira melhora, com a única exceção sendo a vizinhança *relax*. No caso da exploração por primeira melhora, os movimentos candidatos são todos gerados e embaralhados aleatoriamente com o objetivo de remover algum viés causado pela ordem da geração dos vizinhos. Além disso, uma limitação importante é que a busca tabu não é capaz de utilizar a vizinhança *relax*. Visto que o solver realiza a sua própria exploração da vizinhança, a busca tabu perde visibilidade sobre as transições individuais de estados e, por isso, não consegue montar suas estruturas de intensificação e diversificação.

5.1.4 Agendadores

A avaliação da qualidade de uma sequência exige métodos de agendamento precisos, conforme discutido na Seção 2.2.3. Nesse sentido, a literatura do JITJSS recorre frequentemente à programação inteira para converter sequências em agendamentos (Seção 3.1). Por isso, o espaço de projeto contempla esta abordagem por meio da integração com o solver CPLEX. O modelo submetido ao solver fixa a ordem das máquinas e permite ao solver apenas determinar as posições temporais de cada operação.

O projeto integra, adicionalmente, um algoritmo de agendamento utilizado na minimização do *makespan* no problema JSS. Esse algoritmo consiste em agendar as operações o mais cedo possível, com complexidade linear. Porém, como discutido na Seção 2.2.3, essa forma de agendamento tende a gerar penalidades de adiantamento desnecessárias. Para mitigar esse efeito, o sistema adota a lógica de Ferreira [2023]. Após realizar o agendamento da forma descrita, o algoritmo percorre a solução, atrasando operações adiantadas que possuem tempo ocioso subsequente. O algoritmo restringe esse deslocamento a cenários que não impactam operações no prazo ou em atraso, garantindo que o procedimento preserve ou melhore o valor da função objetivo em todos os casos.

5.2 Configuração automática

Como proposto na metodologia e descrito no referencial teórico, agora que temos o espaço de projeto construído, precisamos formalizá-lo de forma que seja possível parametrizar a geração automática de algoritmos. Para isso, utilizaremos uma gramática livre de contexto.

5.2.1 Gramática e parâmetros

A Figura 5.6 contém a gramática que gera todas as combinações algorítmicas que o espaço de projeto admite. Todas as meta-heurísticas precisam de uma solução inicial (*sol_ini*), que pode ser o algoritmo de GT (*gt*) ou o algoritmo construtivo comum (*constr*). Ambas as soluções iniciais precisam de uma regra de despacho (*regra_desp*) que pode ser aleatória (*aleatoria*), a Data de Entrega mais Próxima (do inglês, *Earliest Due Date*) (*EDD*) (*edd*) ou a regra ALL+CR+SPT (*acs*).

Outro símbolo que independe das meta-heurísticas selecionadas é o agendador (*agendador*) para que o algoritmo seja capaz de avaliar a qualidade das sequências. O algoritmo é capaz de realizar este processo por meio de programação inteira (*cplex*)

Figura 5.6. Gramática livre de contexto para geração dos algoritmos

```

    <alg> ::= <sol_ini> <busca> <agendador>
    <sol_ini> ::= constr(<regra_desp>) | gt(<regra_desp>)
    <regra_desp> ::= edd | acs | aleatoria
    <agendador> ::= cplex | linear
    <busca> ::= tabu(<vizinhanca>, <tenure>, <tam_lista_elite>,
        <limite_iters_elite>, <div_decresc>,
        <max_d>, <max_c>)|
        ls(<vizinhanca>)|
        ils(<busca>, <vizinhanca>, <vizinhancas>)
    <vizinhancas> ::= <vizinhanca> <vizinhancas>
    <vizinhancas> ::= ""
    <vizinhanca> ::= <tipo_vizinhanca> (<melhora>)
    <tipo_vizinhanca> ::= relax | troca_al | troca_penal | troca_adj |
        troca_crit | insere_al | insere_penal
    <melhora> ::= mm | pm
    <tenure> ::= [10,100]
    <max_d> ::= [1, 100]
    <max_c> ::= [1, 10]
    <tam_lista_elite> ::= [5, 30]
    <limite_iters_elite> ::= [100, 3000]
    <div_decresc> ::= [2, 20]

```

Fonte: Os Autores.

ou via procedimento linear que agenda as operações o mais cedo possível, mas com atraso deliberado para mitigação de penalidades de adiantamento (**linear**).

Após a derivação da solução inicial, da regra de despacho e do agendador, a gramática induz a escolha do método de busca (**busca**). Este pode ser uma Busca Local (do inglês, *Local Search*) (**LS**) (**ls**), uma Busca Tabu (do inglês, *Tabu Search*) (**TS**) (**tabu**) ou uma **ILS** (**ils**). Os métodos de busca **ls** e **tabu** precisam apenas de uma vizinhança, que pode ser escolhida entre a **relax**, **troca_al**, **troca_penal**, **troca_adj**, **troca_crit**, **insere_al** e **insere_penal**. Por outro lado, a **ils** pode receber um número arbitrário de vizinhanças, e cada uma pode ser explorada por primeira melhora (**pm**) ou melhor melhora (**mm**).

A escolha do **tabu** como método de busca exige a definição de símbolos adicionais, como o tamanho da lista tabu (**tenure**). Os parâmetros da lista de elementos elite: o tamanho da lista (**tam_list_elite**), o número máximo de iterações para que um elemento da lista seja recuperado (**limite_iters_elite**) e o número que divide o limite de iterações quando nenhuma melhoria é encontrada (**div_decresc**). Já o detector de ciclos requer o ajuste do tamanho do ciclo (**max_d**) e do limite de repetições do mesmo ciclo para identificação de estagnação (**max_c**).

Por fim, se escolhermos a **ils** como processo de busca, é necessário selecionar

outro método de busca para ser utilizado internamente. Este caráter recursivo obriga o sistema a derivar novos símbolos idênticos aos anteriores até a obtenção de uma sequência de terminais formando um algoritmo executável.

Porém, como descrito na Seção 2.4.2, não podemos utilizar a gramática da Figura 5.6 em um algoritmo configurador, pois essa gramática gera infinitas palavras e o algoritmo configurador espera um espaço de projeto finito. Dessa forma, a realização da configuração automática exige uma conversão dessa gramática para um conjunto de parâmetros que force a finitude do espaço de projeto. A Tabela 5.2 detalha o resultado da conversão. Em relação à gramática, os parâmetros representam símbolos não terminais que precisam ser derivados para os possíveis valores. Além disso, a configuração do algoritmo configurador permite utilizar condições que auxiliam na limitação e controle dos algoritmos que podem ser instanciados. Os parâmetros apenas categóricos e que já eram finitos continuam os mesmos: solução inicial, regras de despacho e agendadores.

Em relação aos métodos de busca, a configuração incorpora parâmetros adicionais para a gestão da recursão limitando o nível de recursão da ILS a 1 para evitar estruturas excessivamente complexas. Para controlar esse nível de recursão, um dos parâmetros adicionais é o segundo parâmetro de busca `busca2`. Esse parâmetro só pode ser definido quando o parâmetro `busca1` é igual a `ils` e não pode ser igual a `ils`. Dessa forma, uma ILS não pode ser definida como processo de busca interno de outra ILS.

Outro parâmetro é a quantidade de vizinhanças. Apenas a ILS permite múltiplas vizinhanças, então esse parâmetro só fica ativo quando a primeira busca é ILS. O intervalo desse parâmetro permite até três vizinhanças na ILS. Junto a isso, para que o irace consiga definir as vizinhanças utilizadas e seu modo de exploração, o espaço de projeto disponibiliza três parâmetros de vizinhanças (`vizx` e `viz_tabux`), cuja ativação depende do valor atribuído a `num_viz`. Além disso, cada um dos parâmetros de vizinhança possui associado a ele um parâmetro que determina o tipo de exploração daquela vizinhança (`viz_explx`)

O último parâmetro de controle da ILS não está tão relacionado a adaptação da gramática. O parâmetro de duração das iterações da ILS (`max_milli_ils_iter`) controla o quanto o processo de busca deve continuar explorando a vizinhança e permite ao irace equilibrar a intensificação com a diversificação da ILS.

Apesar de todos esses parâmetros de controle da ILS, é importante mencionar que ainda seria possível que o algoritmo configurador executasse a ILS com duas ou três vizinhanças iguais. Não é possível limitar essa escolha apenas com a configuração representada na Tabela 5.2. Para melhorar o cenário de configuração, o processo emprega o mecanismo de reparação de configurações do pacote irace via *script* em R, garantindo a integridade dos algoritmos gerados. O configurador submete cada configuração

Tabela 5.2. Espaço de busca de parâmetros do irace.

Parâmetro	Valores	Condição
sol_ini	gt, constr	-
regra_desp	edd, acs, aleatoria	-
busca ₁	ls, ils, tabu	-
busca ₂	ls, tabu	busca ₁ = 'ils'
max_milli_ils_iter	[1000, 30000]	busca ₁ = 'ils'
num_viz	[1, 3]	busca ₁ = 'ils'
viz ₁	Vizinhanças LS (ver legenda)	busca ₁ = 'ls' OR 'busca ₂ = 'ls'
viz_tabu ₁	Vizinhanças TABU (ver legenda)	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
viz_expl ₁	mm, pm	viz ₁ ≠ 'relax'
viz ₂	Vizinhanças LS (ver legenda)	num_viz ≥ 2 AND busca ₁ = 'ils' AND busca ₂ = 'ls'
viz_tabu ₂	Vizinhanças TABU (ver legenda)	num_viz ≥ 2 AND busca ₁ = 'ils' AND busca ₂ = 'tabu'
viz_expl ₂	mm, pm	num_viz ≥ 2 AND viz ₂ ≠ 'relax'
viz ₃	Vizinhanças LS (ver legenda)	num_viz ≥ 3 AND busca ₁ = 'ils' AND busca ₂ = 'ls'
viz_tabu ₃	Vizinhanças TABU (ver legenda)	num_viz ≥ 3 AND busca ₁ = 'ils' AND busca ₂ = 'tabu'
viz_expl ₃	mm, pm	num_viz ≥ 3 AND viz ₃ ≠ 'relax'
agendador	linear, cplex	-
max_d	[5, 100]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
max_c	[1, 10]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
tenure	[10, 100]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
tam_list_elite	[5, 30]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
limite_iters_elite	[100, 3000]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'
div_decresc	[2, 20]	busca ₁ = 'tabu' OR 'busca ₂ = 'tabu'

Vizinhanças LS: troca_adj, troca_al, troca_penal, troca_crit, insere_al, insere_penal, relax; **Vizinhanças TABU:** troca_adj, troca_al, troca_penal, troca_crit, insere_penal, insere_al; **Fonte:** Os Autores.

gerada ao *script* de reparo, que identifica vizinhanças duplicadas e as substitui por componentes ainda não selecionados.

O conjunto de parâmetros numéricos da busca tabu mantém a fidelidade à gramática original, sob a condição de ativação do modo `tabu`.

Por fim, também relacionado à `TS`, cada parâmetro de vizinhança aparece duplicado. Isso acontece pois, como mencionado anteriormente, a busca tabu não suporta a vizinhança `relax` e, por isso, essa vizinhança só está disponível quando um dos dois parâmetros de busca for igual a `ls`.

5.2.2 Instâncias de treino

A obtenção de resultados precisos na configuração automática exige a separação entre os conjuntos de instâncias de treino e de teste, conforme a fundamentação da Seção 2.3.4. Para isso, a metodologia estabelece a geração de novas instâncias para a composição do conjunto de treinamento. O procedimento de geração segue rigorosamente as etapas detalhadas na Seção 4.1. Além disso, o processo de validação submete cada instância gerada ao solver CPLEX por um período de uma hora, sob paralelismo de 4 núcleos. O critério de seleção descarta automaticamente instâncias para as quais o solver localiza a solução ótima dentro do limite temporal estabelecido. Essa estratégia resulta em um conjunto de treinamento composto por instâncias que desafiam a capacidade de resolução de modelos de programação inteira em tempo hábil.

Como descrito na metodologia, o conjunto de testes possui 72 instâncias com diferentes combinações de tamanhos. Nesse sentido, a etapa de geração produziu 72 instâncias, respeitando a paridade de tamanhos e combinações em relação ao conjunto de teste descrito na metodologia.

5.2.3 Execução do *irace*

A integração do algoritmo configurável, do conjunto de treinamento e da definição do espaço de busca no pacote *irace* consolida os elementos fundamentais para o início do processo de configuração. Para completar essa coleção, o processo estabelece um orçamento de configuração de 25.000 execuções para o algoritmo. Cada execução limita-se ao tempo de 60 segundos, critério baseado no desempenho médio dos algoritmos de estado da arte. Além disso, o sistema executa o *irace* de forma paralela em 6 *threads*. Em contrapartida, as chamadas ao solver CPLEX, tanto em vizinhanças quanto em agendamentos, operam estritamente sem paralelismo para garantir a integridade dos testes de desempenho individual.

O repositório Leles [2026] disponibiliza o código-fonte, *scripts*, instâncias e as configurações completas utilizadas no experimento.

5.3 Análise

A execução do pacote irace sobre o espaço de projeto detalhado anteriormente executou 10 iterações por aproximadamente 60 horas, utilizando cerca de 360 horas de processamento total, quando somados os tempos de uso de cada núcleo de processamento.

Organizaremos os resultados deste trabalho em duas subseções. Na primeira, argumentaremos a validade e a confiabilidade dos resultados do processo de calibração. Na segunda subseção, apresentaremos os testes das configurações no conjunto de instâncias de teste para comparação com as soluções do estado da arte.

5.3.1 Validade do processo de configuração

Primeiramente, a Tabela 5.3 apresenta as cinco configurações elite selecionadas pelo pacote irace como as mais eficazes para a conjunto de instâncias proposto. Nessa tabela, cada coluna representa uma das 5 melhores configurações encontradas pelo irace. Cada linha representa um parâmetro do algoritmo e os valores que cada configuração define para ele. A tabela omite os parâmetros não instanciados pelo configurador para este conjunto específico de elite.

Tabela 5.3. Configurações Elite Finais

Parâmetro	C1	C2	C3	C4	C5
sol_ini	gt	gt	gt	gt	gt
regra_disp	edd	edd	edd	edd	edd
busca ₁	ils	ils	ils	ils	ils
busca ₂	tabu	tabu	tabu	tabu	tabu
max_milli_ils_iter	19900	21254	9713	5741	14594
num_viz	2	2	3	3	3
nhood_tabu ₁	troca_penal	troca_penal	troca_penal	troca_al	troca_penal
viz_expl ₁	pm	pm	pm	mm	pm
nhood_tabu ₂	troca_crit	troca_crit	troca_crit	troca_adj	troca_crit
viz_expl ₂	mm	pm	pm	pm	pm
nhood_tabu ₃	-	-	troca_al	troca_penal	troca_al
viz_expl ₃	-	-	pm	mm	pm
agendador	linear	linear	linear	linear	linear
max_d	29	19	44	65	38
max_c	10	2	3	3	2
tenure	98	96	98	70	85
tam_list_elite	23	22	21	13	21
limite_iters_elite	2788	2164	1935	2749	1353
div_decresc	16	12	8	10	11

Fonte: Os Autores.

A análise das configurações revela uma uniformidade significativa nos parâmetros de alto nível, como o método de busca e o agendador. Por exemplo, os parâmetros categóricos `sol_ini`, `regra_disp`, `busca1`, `busca2` e `agendador` são iguais entre todas

as configurações. De maneira similar, os parâmetros numéricos demonstram a concentração da busca em faixas de valores específicas, como ilustra o parâmetro `tenure` que reside no intervalo $[70, 98]$ em todas as configurações elite.

Além disso, as escolhas referentes às vizinhanças também apresentam uma baixa diversidade estrutural. O processo de configuração descartou as vizinhanças `insere_al`, `insere_penal` e `relax` em favor de melhores componentes para o cenário de configuração. Ademais, todas as configurações elite utilizam as vizinhanças `troca_penal` e quatro das cinco utilizam `troca_crit`, indicando que a exploração do espaço de busca focada no caminho crítico e em penalidades individuais é determinante para a performance na resolução do [JITJSS](#).

Essa homogeneidade entre as configurações elite evidencia a convergência do modelo do *irace* para uma região de alto desempenho no espaço de projeto. A redução e estabilização da variação entre as configurações elite indicaria que o modelo do *irace* concorda que múltiplas configurações produzem soluções de qualidade. Contudo, a avaliação visual não quantifica com precisão a variância interna desse grupo. Para essa finalidade, o estudo emprega o Coeficiente de Distância de Gower [Gower, 1971]. Este coeficiente quantifica a dissimilaridade entre indivíduos com dados de naturezas distintas, utilizando o intervalo total do espaço de busca para a normalização das variáveis numéricas. No contexto deste trabalho, as variáveis abrangem parâmetros categóricos e numéricos.

$$d(i, j) = \frac{\sum_{k=1}^n \delta_{ij}^{(k)} s_{ij}^{(k)}}{\sum_{k=1}^n \delta_{ij}^{(k)}} \quad (5.1)$$

O cálculo da distância de Gower ocorre de acordo com a Equação 5.1. Nesta fórmula, n representa o número total de parâmetros do algoritmo. Além disso, $s_{ij}^{(k)}$ representa a similaridade entre as configurações i e j para o parâmetro k , sendo que:

- Para variáveis categóricas: $s = 0$ se $x_{ik} = x_{jk}$, e $s = 1$ caso contrário;
- Para variáveis numéricas: $s = \frac{|x_{ik} - x_{jk}|}{R_k}$, sendo R_k a amplitude do parâmetro k .

Por fim, a variável $\delta_{ij}^{(k)}$ assume o valor 1 quando ambos os parâmetros existem nas configurações comparadas e 0 em caso de ausência.

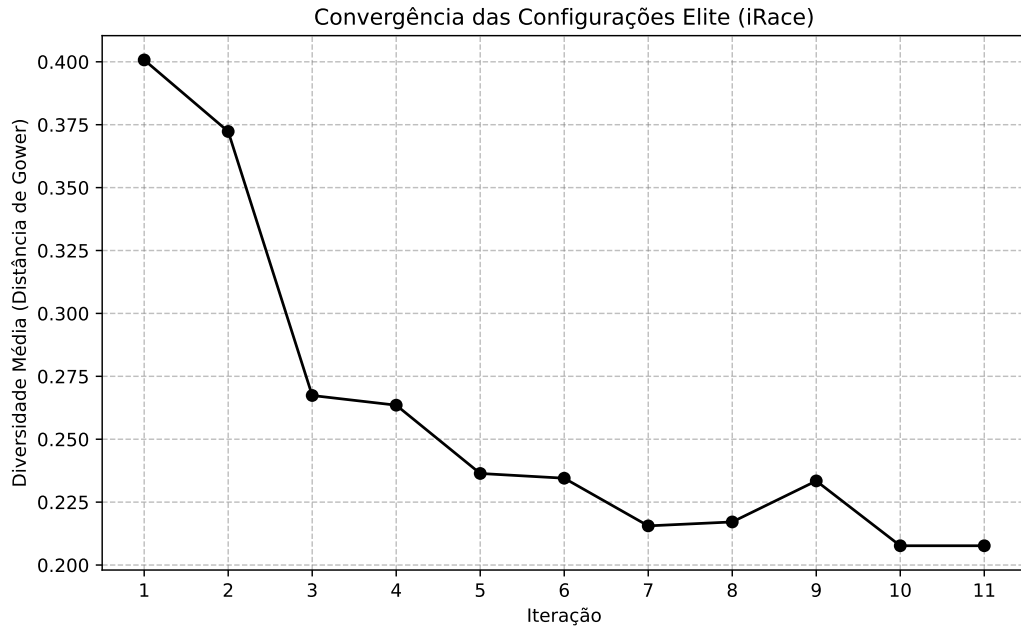
Contudo, a aplicação direta do coeficiente de Gower apresenta limitações na representação da semelhança funcional entre os algoritmos. Por conta das múltiplas vizinhanças, a distância de Gower superestimaria a dissimilaridade entre uma permutação diferente das mesmas três vizinhanças. Porém, devido à natureza circular do percorrido das vizinhanças, diferentes permutações de um mesmo conjunto de vizinhanças resultam em comportamentos algorítmicos funcionalmente similares.

Para mitigar essa distorção, utilizamos o Coeficiente de Distância de Jaccard [Jaccard, 1901] para quantificar a semelhança entre os conjuntos de vizinhanças, desconsiderando a ordem de derivação. Ele é dado pela Equação 5.2 e consiste na divisão entre o tamanho da interseção de dois conjuntos pelo tamanho da união dos dois conjuntos.

$$Jaccard(U, V) = 1 - \frac{|U \cap V|}{|U \cup V|} \quad (5.2)$$

Dessa forma, para cada uma das 11 iterações do irace, calculamos a média de $d(i, j)$ para cada um dos pares formados pelas 5 configurações elite de cada iteração. O valor dessa média das distâncias entre as configurações pode ser visto no gráfico da Figura 5.7.

Figura 5.7. Gráfico da variância da distância de Gower entre as configurações elite pelas iterações do irace.



Fonte: Os Autores.

O gráfico da Figura 5.7 demonstra a redução progressiva da variância entre as configurações elite ao longo das iterações. Nesse sentido, a estabilização da variância na segunda metade do processo confirma a convergência do modelo de parâmetros para uma região de consenso, indicando que o processo de exploração atingiu maturidade no espaço de projeto.

Antes mesmo de realizar qualquer teste com as configurações elite, a análise das configurações elite permite a correlação direta com a literatura de JITJSS. Por exemplo, a seleção de múltiplas vizinhanças no processo de busca. O processo de configuração

pelo *irace* ratifica a eficácia dessa estratégia, corroborando as abordagens de Ahmadian et al. [2021] e Wang & Li [2014]. Este cenário fornece um forte indício de que a exploração via vizinhança única é insuficiente diante da dificuldade do problema.

Além disso, diferentemente dos trabalhos citados, o presente trabalho propõe arquiteturas baseadas em uma Busca Tabu Iterada. Ou seja, o processo de configuração identificou que a integração de mecanismos tabu proporciona ganhos de desempenho à busca, mesmo em estruturas de múltiplas vizinhanças.

A predominância da solução inicial via [GT](#), associada à regra `edd`, reafirma a robustez desta combinação para o [JITJSS](#).

5.3.2 Teste das configurações

A convergência do processo de configuração para um grupo de algoritmos com características análogas fundamenta a etapa de testes em instâncias da literatura e a posterior comparação com o estado da arte. Para isso, a métrica de qualidade para cada instância consiste no desvio percentual em relação ao melhor valor conhecido da instância, conforme estabelecido pela Equação 5.3. Nela, S corresponde ao nosso resultado e M ao melhor resultado conhecido para a instância. Ou seja, nesta escala, valores positivos indicam desempenho inferior à literatura, enquanto valores negativos apontam a superação dos marcos registrados.

$$\frac{S - M}{M} * 100 \quad (5.3)$$

O procedimento inicial envolve a aplicação das configurações elite sobre o conjunto de teste. Por conta dos componentes estocásticos presentes nos algoritmos, geramos 5 *seeds* aleatórias e realizamos a execução de cada configuração 5 vezes em cada instância, utilizando uma *seed* diferente em cada execução.

A Tabela 5.4 contém um resumo do desempenho de cada configuração elite em comparação com o estado da arte. Cada coluna dessa tabela representa uma das configurações elite e as linhas contém: o número de instâncias nas quais o melhor resultado entre as cinco execuções é igual ou melhor ao estado da arte e o número de instâncias nas quais a mediana entre as cinco execuções é melhor ou igual ao estado da arte. Com isso, como representado na tabela, a configuração C1 demonstrou maior capacidade de generalização ao atingir o maior volume de resultados melhores ou iguais ao estado da arte no conjunto de teste (52 instâncias no total).

Dada a equivalência de desempenho entre os indivíduos elite, as análises subsequentes concentram-se na configuração C1. As tabelas 5.5, 5.6 e 5.7 detalham os resultados para instâncias de 10, 15 e 20 tarefas, respectivamente, estabelecendo o comparativo

Tabela 5.4. Resumo dos resultados das configurações elite.

Parâmetro	C1	C2	C3	C4	C5
Número de instâncias nas quais a melhor das soluções geradas pela configuração C1 é melhor ou igual ao estado da arte	52	48	47	43	48
Número de instâncias nas quais a mediana das soluções geradas pela configuração C1 é melhor ou igual ao estado da arte	32	26	30	32	28

Fonte: Os Autores.

Tabela 5.5. Resultados da configuração C1 para as instâncias de teste de 10 tarefas.

Instância	Cplex	[1]	[2]	[3]	[4]	[5]	B	M
loose-equal-1-10x2	224,84	224,84	224,84	224,84	224,84	224,84	224,84	224,84
loose-equal-2-10x2	319,37	324,43	319,37	319,37	319,37	319,37	319,37	319,37
loose-equal-1-10x5	1738,05	1733,38	1738,05	1719,63	1725,16	1719,63	1719,63	1738,63
loose-equal-2-10x5	971,96	968,89	967,73	967,73	968,05	967,73	967,63	967,73
loose-equal-1-10x10	360,74	366,77	365,81	364,39	360,56	360,33	360,74	364,59
loose-equal-2-10x10	251,86	249,85	252,99	249,85	249,85	249,85	249,85	249,85
loose-tard-1-10x2	416,44	416,44	416,44	416,44	416,44	416,44	416,44	416,44
loose-tard-2-10x2	137,94	137,94	137,94	137,94	137,94	137,94	137,94	137,94
loose-tard-1-10x5	176,83	175,08	175,08	175,08	175,08	175,08	175,08	175,08
loose-tard-2-10x5	492,05	525,88	499,93	485,06	488,6	486,93	485,06	485,06
loose-tard-1-10x10	375,71	383,84	380,26	374,2	374,2	375,58	374,2	386,4
loose-tard-2-10x10	144,94	144,94	144,99	144,94	144,94	144,94	144,94	144,94
tight-equal-1-10x2	461,96	461,96	461,96	461,96	461,96	461,96	461,96	461,96
tight-equal-2-10x2	448,32	448,32	448,32	448,32	448,32	448,32	448,32	448,32
tight-equal-1-10x5	689,11	689,11	689,11	689,11	689,11	689,11	689,11	690,29
tight-equal-2-10x5	763,24	770,49	764,03	763,24	763,24	763,24	763,24	763,24
tight-equal-1-10x10	1281,66	1276,23	1277,44	1276,23	1276,23	1279,82	1281,66	1301,46
tight-equal-2-10x10	1885,25	1885,25	1871,23	1866,92	1865,94	1886,93	1865,93	1873,64
tight-tard-1-10x2	179,46	179,46	179,46	179,46	179,46	179,46	179,46	179,46
tight-tard-2-10x2	145,37	145,37	145,37	145,37	145,37	145,37	145,37	145,37
tight-tard-1-10x5	379,19	380,82	380	379,19	379,19	379,19	386,57	389,43
tight-tard-2-10x5	635,93	627,45	628,68	627,45	628,02	627,45	627,45	627,45
tight-tard-1-10x10	687,65	717,85	746,03	668,14	667,97	669,15	669,93	687,65
tight-tard-2-10x10	779,17	780,82	785,79	777,85	778,32	780,74	780,66	804,23

Fonte: A coluna Cplex foi retirada de Ahmadian et al. [2021] e cada coluna numerada representa os resultados de um trabalho: [1] Dos Santos et al. [2010], [2] Wang & Li [2014], [3] Ahmadian et al. [2021], [4] Abderrazzak et al. [2022] e [5] Sabri et al. [2023].

com os resultados da literatura. Essas tabelas identificam as instâncias na primeira coluna e cada uma das colunas subsequentes contém os resultados dos diferentes trabalhos que compõem o estado da arte. Por fim, as duas últimas colunas representam os resultados do presente trabalho de forma que B representa o melhor resultado obtido entre cinco execuções da configuração e a coluna M representa a mediana dos resultados obtidos entre cinco execuções. Os resultados que representam o melhor valor encontrada para a respectiva instância são destacados em negrito nessas tabelas.

Tabela 5.6. Resultados da configuração C1 para as instâncias de teste de 15 tarefas.

Instância	Cplex	[1]	[2]	[3]	[4]	[5]	B	M
loose-equal-1-15x2	1041,33	1041,33	1041,33	1041,33	1041,33	1041,33	1041,33	1041,33
loose-equal-2-15x2	497,97	497,97	514,72	512,54	497,97	497,97	497,97	505,16
loose-equal-1-15x5	3457,57	3220,16	3321,54	3272,46	3258,72	3229,16	3196,85	3247,89
loose-equal-2-15x5	3437,32	3328,86	3310,27	3260,81	3255,13	3251,84	3274,38	3290,70
loose-equal-1-15x10	875,74	1008,28	977,43	906,49	897,51	913,26	898,99	938,80
loose-equal-2-15x10	1587,09	1720,18	1584,44	1487,33	1538,27	1492,11	1554,18	1594,60
loose-tard-1-15x2	654,84	654,84	654,84	654,84	654,84	654,84	654,84	654,84
loose-tard-2-15x2	293,17	279,71	297,54	285,16	289,36	279,71	279,71	290,88
loose-tard-1-15x5	1281,87	1320,93	1315,47	1281	1292,28	1317,56	1280,74	1281,41
loose-tard-2-15x5	335,55	329,47	335,55	328,26	330,14	332,25	327,60	328,88
loose-tard-1-15x10	277	297,19	296,6	277	277	277	281,45	287,03
loose-tard-2-15x10	597,93	713,97	664,13	601,24	627,81	603,67	587,24	592,31
tight-equal-1-15x2	3366,66	3344,54	3344,54	3344,54	3344,54	3344,54	3344,54	3344,54
tight-equal-2-15x2	1479,76	1479,76	1479,76	1479,76	1479,76	1479,76	1479,76	1479,76
tight-equal-1-15x5	1412,71	1369,06	1345,21	1342,3	1343,08	1345,18	1318,68	1318,68
tight-equal-2-15x5	2782,17	2630,06	2709,13	2641,28	2563,16	2602,99	2626,67	2651,16
tight-equal-1-15x10	7167,59	7380,76	8120,43	6820,87	6811,54	7115,63	6610,44	6748,88
tight-equal-2-15x10	5407,42	4893,44	4966,34	4760,48	4801,47	4757,06	4652,41	4730,08
tight-tard-1-15x2	806,92	790,5	790,66	790,5	790,5	790,5	790,5	790,5
tight-tard-2-15x2	905,37	905,37	905,37	905,37	905,37	905,37	905,37	905,37
tight-tard-1-15x5	1371,29	1371,37	1426,97	1359,18	1366,45	1341,69	1358,21	1359,18
tight-tard-2-15x5	721,46	691,1	725,72	681,27	677,8	695,08	677,57	681,62
tight-tard-1-15x10	815,03	800,87	832,08	767,26	770,91	775,4	757,83	785,75
tight-tard-2-15x10	1267,53	1310,04	1452,9	1224,82	1243,52	1226,23	1170,94	1172,30

Fonte: A coluna Cplex foi retirada de Ahmadian et al. [2021] e cada coluna numerada representa os resultados de um trabalho: [1] Dos Santos et al. [2010], [2] Wang & Li [2014], [3] Ahmadian et al. [2021], [4] Abderrazzak et al. [2022] e [5] Sabri et al. [2023].

A análise dos dados revela que o algoritmo configurado igualou ou superou o estado da arte em aproximadamente 72,2% das 72 instâncias de teste. Essa eficácia concentra-se predominantemente nas instâncias de 10 tarefas, nas quais o algoritmo atingiu o marco de 83% de sucesso em relação aos melhores valores da literatura. Além disso, a análise das medianas corrobora a robustez da configuração C1, a qual igualou ou superou os marcos bibliográficos em 44,4% das instâncias.

Os dados expostos indicam uma redução gradual na eficácia da mediana conforme a escala das instâncias aumenta. Contudo, o desvio médio das medianas em relação às melhores soluções individuais é de apenas 1,06%, evidenciando a estabilidade do método estocástico frente a variações de sementes aleatórias.

Embora os marcos de desempenho demonstrem competitividade, a análise prévia baseia-se no melhor valor conhecido para cada instância. Esta métrica representa um limite superior composto pelos melhores resultados de todos os trabalhos da literatura combinados. Dessa forma, a comparação individual apresentada na Tabela 5.8 isola o desempenho da proposta frente a cada método do estado da arte de maneira independente. As colunas organizam os resultados por autor, enquanto as linhas quantificam o número de instâncias nas quais a configuração C1 obteve desempenho igual ou superior

Tabela 5.7. Resultados da configuração C1 para as instâncias de teste de 20 tarefas.

Instância	Cplex	[1]	[2]	[3]	<i>B</i>	<i>M</i>
loose-equal-1-20x2	2565,51	2548,95	2551,22	2548,95	2548,95	2549,19
loose-equal-2-20x2	3105,34	3069,13	3070,16	3069,13	3069,13	3069,13
loose-equal-1-20x5	8271,34	7545,44	7758,02	7524,13	7487,77	7666,29
loose-equal-2-20x5	7793,45	7252,81	7302,01	7059,38	7022,53	7128,87
loose-equal-1-20x10	5597,4	5260,48	5476,47	4651,31	4592,02	4687,60
loose-equal-2-20x10	1862,41	1987,65	1718,08	1597,89	1608,61	1701,42
loose-tard-1-20x2	1265,36	1206,97	1205,59	1206,4	1203,88	1206,97
loose-tard-2-20x2	772,89	769,35	790,12	769,35	771,06	776,01
loose-tard-1-20x5	3548,14	2938,24	3517,2	2931,11	2936,63	3068,78
loose-tard-2-20x5	4048,77	3722,61	3684,67	3627,51	3617,49	3646,58
loose-tard-1-20x10	6494,34	5093,89	5478,21	4817,32	4789,13	5114,72
loose-tard-2-20x10	1545,95	1570,64	1599,55	1401,28	1516,95	1531,54
tight-equal-1-20x2	1940,06	1937,49	1939,11	1936,45	1933,72	1933,72
tight-equal-2-20x2	961,44	941,69	959,65	943,7	944,93	948,01
tight-equal-1-20x5	3013,79	2866,94	2935,95	2865,95	2821,02	2865,45
tight-equal-2-20x5	7233,35	6741,16	6964,91	6613,62	6581,72	6626,55
tight-equal-1-20x10	11561,7	10607,2	11073,1	10378,8	10534,50	10831,90
tight-equal-2-20x10	8501,42	7310,32	7317,74	6839,3	6892,16	7047,16
tight-tard-1-20x2	1675,37	1671,87	1682,94	1674,6	1671,87	1672,03
tight-tard-2-20x2	1489,65	1448,66	1459,57	1451,61	1448,66	1448,66
tight-tard-1-20x5	3821,29	3672,18	3669,09	3620,61	3598,12	3618,13
tight-tard-2-20x5	1881,02	1858,19	1976,11	1792,78	1802,78	1808,36
tight-tard-1-20x10	4984,31	4853,81	5536,64	4445,59	4406,01	4434,02
tight-tard-2-20x10	4057,01	3358,61	3245,9	3085,54	3207,33	3242,14

Fonte: A coluna Cplex foi retirada de Ahmadian et al. [2021] e cada coluna numerada representa os resultados de um trabalho: [1] Dos Santos et al. [2010], [2] Wang & Li [2014] e [3] Ahmadian et al. [2021].

ao de cada método de referência. Com isso, os resultados evidenciam que a configuração gerada por este estudo apresenta qualidade superior em comparações diretas com cada trabalho da literatura.

Como mencionado anteriormente, todas as execuções do algoritmo levam 60 segundos. O algoritmo de ILS opera sem mecanismos de detecção de convergência das soluções, gastando o tempo limite de 60 segundos em todas as execuções. Entretanto, a localização da solução final ocorre, em média, aos 21,1 segundos de execução. Em instâncias de 10 tarefas, este marco reduz-se para 9 segundos. Esses indicadores sugerem que a integração de critérios de convergência otimizará a eficiência computacional do algoritmo, permitindo a obtenção de resultados da mesma qualidade em frações do tempo total estabelecido.

Tabela 5.8. Comparação individual entre este trabalho e algoritmos da literatura.

	[1]	[2]	[3]	[4]	[5]
Porcentagem das instâncias nas quais a melhor das soluções geradas pela configuração C1 é melhor ou igual aos resultados de um trabalho	94,44	97,22	77,77	79,16	81,25
Porcentagem das instâncias nas quais a mediana das soluções geradas pela configuração C1 é melhor ou igual aos resultados de um trabalho	75	88,88	50	62,5	62,5

Fonte: cada coluna numerada representa os resultados de um trabalho: [1] Dos Santos et al. [2010], [2] Wang & Li [2014], [3] Ahmadian et al. [2021], [4] Abderrazzak et al. [2022] e [5] Sabri et al. [2023].

Capítulo 6

Conclusão

Neste trabalho, apresentamos a aplicação das técnicas de AAC e AAD para a resolução do problema JITJSS. A motivação central fundamentou-se na eliminação do viés humano e do esforço mecânico associados ao desenvolvimento manual de heurísticas, propondo, em contrapartida, um espaço de projeto estruturado e explorado de forma sistemática pelo pacote *irace*.

Nossas contribuições incluem a nova vizinhança `troca_crit` para o problema JITJSS, construída a partir do estudo das componentes da literatura. Com isso, a partir da aplicação das técnicas de AAD, constatamos que a inclusão no espaço de projeto dessa vizinhança, focada no caminho crítico individual de operações atrasadas, mostrou-se determinante, sendo integrada pelo configurador em 80% das soluções de elite.

Além disso, como resultado do processo de AAD, propomos um novo algoritmo estado da arte para o JITJSS. O algoritmo proposto superou ou igualou o estado da arte da literatura em 72,2% das 72 instâncias de teste. Adicionalmente, ao analisarmos a eficiência temporal, observamos que, embora o limite de tempo estivesse fixado em 60 segundos, o algoritmo encontrou a solução final, em média, aos 21,1 segundos de execução, evidenciando uma rápida velocidade de convergência.

Dessa forma, concluímos que os experimentos conduzidos validaram a hipótese inicial de que o projeto automático é capaz de derivar algoritmos competitivos. Outrossim, os experimentos demonstraram que as componentes algorítmicas propostas são particularmente efetivas gerando algoritmos que superam o estado da arte na maioria das instâncias. Nessa perspectiva, o processo de configuração utilizando o *irace* convergiu para uma região de alto desempenho, resultando na configuração elite C1, caracterizada como uma Busca Tabu Iterada.

Como limitações da abordagem atual, apontamos a ausência de um critério de parada antecipada na heurística de busca e a restrição do nível de recursão da gramática

para evitar a complexidade excessiva no processo de amostragem do configurador.

Para trabalhos futuros, listamos as seguintes frentes de pesquisa:

- **Ampliação do Espaço de Projeto:** Inclusão de novos operadores de busca de solução única, como Algoritmos Genéticos, Busca em Grandes Vizinhanças ou Recozimento Simulado, diretamente como blocos terminais da gramática;
- **Análise de Instâncias de Larga Escala:** Aplicação do algoritmo gerado sobre novos conjuntos de instâncias com dimensões superiores a 20 tarefas e 10 máquinas para avaliar o comportamento do algoritmo sob estresse computacional.
- **Aplicação sobre novos domínios:** Aplicação do algoritmo gerado em novos problemas com propriedades similares ao [JITJSS](#). A exemplo, temos o problema *Just-In-Time Job Shop with Setup Times* que é idêntico ao [JITJSS](#), mas adicionando tempos de troca entre operações processadas na mesma máquina.

Referências Bibliográficas

- Abderrazzak, S.; Hamid, A. & Omar, S. (2022). Adaptive large neighborhood search for the just-in-time job-shop scheduling problem. In *2022 International Conference on Control, Automation and Diagnosis (ICCAD)*, pp. 1–6. IEEE.
- Ahmadian, M. M.; Salehipour, A. & Cheng, T. C. E. (2021). A meta-heuristic to solve the just-in-time job-shop scheduling problem. *European Journal of Operational Research*, 288(1):14–29.
- Alfaro-Fernández, P.; Ruiz, R.; Pagnozzi, F. & Stützle, T. (2020). Automatic algorithm design for hybrid flowshop scheduling problems. *European Journal of Operational Research*, 282(3):835–845.
- Araujo, R. P.; dos Santos, A. G. & Arroyo, J. E. C. (2009). Genetic algorithm and local search for just-in-time job-shop scheduling. In *2009 IEEE Congress on Evolutionary Computation*, pp. 955–961. IEEE.
- Aydın, D.; Yavuz, G. & Stützle, T. (2017). Abc-x: a generalized, automatically configurable artificial bee colony framework. *Swarm Intelligence*, 11(1):1–38.
- Baptiste, P.; Flamini, M. & Sourd, F. (2008). Lagrangian bounds for just-in-time job-shop scheduling. *Computers & Operations Research*, 35(3):906–915.
- Bezerra, L. C.; Manuel, L. et al. (2020). Automatically designing state-of-the-art multi- and many-objective evolutionary algorithms. *Evolutionary computation*, 28(2):195–226.
- Birattari, M. & Kacprzyk, J. (2009). *Tuning metaheuristics: a machine learning perspective*, volume 197. Springer.
- Brum, A. & Ritt, M. (2018). Automatic design of heuristics for minimizing the makespan in permutation flow shops. In *2018 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–8. IEEE.

- Brum, A.; Ruiz, R. & Ritt, M. (2022). Automatic generation of iterated greedy algorithms for the non-permutation flow shop scheduling problem with total completion time minimization. *Computers & Industrial Engineering*, 163:107843.
- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L. & Stein, C. (2022). *Introduction to algorithms*. The MIT press.
- De Souza, M. (2022). *Automatic algorithm configuration: methods and applications*. Doutorado em computação, Universidade Federal do Rio Grande do Sul, Porto Alegre. Disponível em: <https://lume.ufrgs.br/handle/10183/236350>.
- De Souza, M. & Ritt, M. (2018a). Automatic grammar-based design of heuristic algorithms for unconstrained binary quadratic programming. In *European Conference on Evolutionary Computation in Combinatorial Optimization*, pp. 67–84. Springer.
- De Souza, M. & Ritt, M. (2018b). An automatically designed recombination heuristic for the test-assignment problem. In *2018 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–8. IEEE.
- Delahaye, D.; Chaimatanan, S. & Mongeau, M. (2018). Simulated annealing: From basics to applications. In *Handbook of metaheuristics*, pp. 1–35. Springer.
- Dos Santos, A. G.; Araujo, R. P. & Arroyo, J. E. (2010). A combination of evolutionary algorithm, mathematical programming, and a new local search procedure for the just-in-time job-shop scheduling problem. In *Learning and Intelligent Optimization: 4th International Conference, LION 4, Venice, Italy, January 18-22, 2010. Selected Papers 4*, pp. 10–24. Springer.
- Du, J. & Leung, J. Y.-T. (1990). Minimizing total tardiness on one machine is np-hard. *Mathematics of Operations Research*, 15(3):483–495.
- Eggenesperger, K.; Lindauer, M. & Hutter, F. (2019). Pitfalls and best practices in algorithm configuration. *Journal of Artificial Intelligence Research*, 64:861–893.
- Ferreira, J. G. (2023). Análise de problemas de agendamento com datas de entrega.
- Flamini, M. & Pacciarelli, D. (2008). Real time management of a metro rail terminus. *European Journal of Operational Research*, 189(3):746–761.
- Franzin, A. & Stützle, T. (2019). Revisiting simulated annealing: A component-based analysis. *Computers & operations research*, 104:191–206.
- Garey, M. R. & Johnson, D. S. (1990). *Computers and Intractability; A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., USA.

- Garey, M. R.; Johnson, D. S. & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of operations research*, 1(2):117–129.
- Giffler, B. & Thompson, G. L. (1960). Algorithms for solving production-scheduling problems. *Operations research*, 8(4):487–503.
- Gower, J. C. (1971). A general coefficient of similarity and some of its properties. *Biometrics*, pp. 857–871.
- Hutter, F.; Hoos, H. H.; Leyton-Brown, K. & Stützle, T. (2009). Paramils: an automatic algorithm configuration framework. *Journal of artificial intelligence research*, 36:267–306.
- Hutter, F.; Lindauer, M.; Balint, A.; Bayless, S.; Hoos, H. & Leyton-Brown, K. (2017). The configurable sat solver challenge (csse). *Artificial Intelligence*, 243:1–25.
- IBM Corp. (2022). Ibm ilog CPLEX optimization studio, versão 22.1.
- Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des alpes et des jura. *Bull Soc Vaudoise Sci Nat*, 37:547–579.
- KhudaBukhsh, A. R.; Xu, L.; Hoos, H. H. & Leyton-Brown, K. (2016). Satenstein: Automatically building local search sat solvers from components. *Artificial Intelligence*, 232:20–42.
- Kleinberg, J. & Tardos, E. (2005). *Algorithm Design*. Addison-Wesley Longman Publishing Co., Inc., USA.
- Laborie, P. & Godard, D. (2007). Self-adapting large neighborhood search: Application to single-mode scheduling problems. *Proceedings MISTA-07, Paris*, 8.
- Lawler, E. L.; Lenstra, J. K.; Kan, A. H. R. & Shmoys, D. B. (1993). Sequencing and scheduling: Algorithms and complexity. *Handbooks in operations research and management science*, 4:445–522.
- Leles, M. P. M. (2026). Configuração e projeto automáticos de algoritmos aplicados ao problema just-in-time job shop scheduling. <https://github.com/maiconpml/aad-jitjs>. Acessado em 03 de maio de 2026.
- Lewis, H. R. & Papadimitriou, C. H. (1997). *Elements of the Theory of Computation*. Prentice Hall PTR, USA, 2nd edição.

- Liao, T.; Stützle, T.; de Oca, M. A. M. & Dorigo, M. (2014). A unified ant colony optimization algorithm for continuous optimization. *European Journal of Operational Research*, 234(3):597–609.
- Liu, S. Q. & Kozan, E. (2011). Scheduling trains with priorities: a no-wait blocking parallel-machine job-shop scheduling model. *Transportation Science*, 45(2):175–198.
- López-Ibáñez, M.; Dubois-Lacoste, J.; Cáceres, L. P.; Birattari, M. & Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58.
- Lourenço, H. R.; Martin, O. C. & Stützle, T. (2018). Iterated local search: Framework and applications. In *Handbook of metaheuristics*, pp. 129–168. Springer.
- Marmion, M.-E.; Mascia, F.; López-Ibáñez, M. & Stützle, T. (2013). Automatic design of hybrid stochastic local search algorithms. In *International workshop on hybrid metaheuristics*, pp. 144–158. Springer.
- Mascia, F.; López-Ibáñez, M.; Dubois-Lacoste, J. & Stützle, T. (2013). From grammars to parameters: Automatic iterated greedy design for the permutation flow-shop problem with weighted tardiness. In *International conference on learning and intelligent optimization*, pp. 321–334. Springer.
- Mascia, F.; López-Ibáñez, M.; Dubois-Lacoste, J. & Stützle, T. (2014). Grammar-based generation of stochastic local search heuristics through automatic algorithm configuration tools. *Computers & operations research*, 51:190–199.
- Monette, J.-N.; Deville, Y. & Van Hentenryck, P. (2009). Just-in-time scheduling with constraint programming. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 19, pp. 241–248.
- Nowicki, E. & Smutnicki, C. (1996). A fast taboo search algorithm for the job shop problem. *Management science*, 42(6):797–813.
- O’Neill, M. E. (2014). Pcg: A family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, Claremont, CA.
- Pagnozzi, F. & Stützle, T. (2019). Automatic design of hybrid stochastic local search algorithms for permutation flowshop problems. *European journal of operational research*, 276(2):409–421.

- Pham, D.-N. & Klinkert, A. (2008). Surgical case scheduling as a generalized job shop scheduling problem. *European Journal of Operational Research*, 185(3):1011–1025.
- Pinedo, M. (2008). *Scheduling: Theory, Algorithms, and Systems*. Springer.
- Sabri, A.; Allaoui, H. & Souissi, O. (2023). Reinforcement learning for the just-in-time job-shop scheduling problem. In *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*, pp. 1367–1372. IEEE.
- Sipser, M. (2013). *Introduction to the Theory of Computation*. Cengage Learning.
- Stützle, T. & López-Ibáñez, M. (2018). Automated design of metaheuristic algorithms. In *Handbook of metaheuristics*, pp. 541–579. Springer.
- Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. Wiley.
- Wang, S. & Li, Y. (2014). Variable neighbourhood search and mathematical programming for just-in-time job-shop scheduling problem. *Mathematical Problems in Engineering*, 2014(1):431325.
- Yang, H. A.; Li, J. Y. & Qi, L. L. (2012). An improved genetic algorithm for just-in-time job-shop scheduling problem. *Advanced materials research*, 472:2462–2467.
- Zubaran, T. & Ritt, M. (2016). A tabu search for the parallel machines job-shop scheduling problem. *Anais do XLVIII Simpósio Brasileiro de Pesquisa Operacional. Vitória, Brazil*.
- Zäpfel, G.; Braune, R. & Bögl, M. (2014). *Metaheuristic Search Concepts: A Tutorial with Applications to Production and Logistics*. Springer.

